

RevertPay v1

Complete Technical Interview Master Handbook

Stack: Spring Boot 4.1.1 · Java 25 · Spring MVC · Spring Data JPA / Hibernate · Spring Security + JWT (jjwt 0.13) · PostgreSQL (Neon) · React 19 + Vite

Source of truth: the supplied repository Siva2583/RevertPay-v-1.0 (commit d5b2dd7) — every statement classified below.

Scope: 25 sections · architecture & flow diagrams · state machine · double-entry ledger · ACID & concurrency · idempotency audit · security trace · 34 failure scenarios · 180 interview questions · 10-page cheat sheet

Tag	Meaning
●	CONFIRMED FROM CODE — read in the supplied source
●	CONFIRMED FROM README/RESUME (this repo has no project README — almost nothing is this class)
●	TECHNICAL INFERENCE — follows from framework behaviour, not literally in code
●	NOT FOUND / DO NOT CLAIM — absent from the implementation; say so honestly

Prepared 29 September 2026 · In diagrams inside code blocks: [OK]=confirmed · [INFER]=inference · [DOC]=docs · [GAP]=not implemented

Every claim below is classified:

Tag	Meaning
●	CONFIRMED FROM CODE — I read the file; the statement is literally in the source
●	CONFIRMED FROM README/RESUME — stated in project docs (note: this repo has no project README ; only the default Vite README — so almost nothing is ●)
●	TECHNICAL INFERENCE — not literally in code, but follows from framework behaviour (e.g., what Hibernate <code>ddl-auto=update</code> generates)
●	NOT FOUND / DO NOT CLAIM — absent from the supplied implementation; saying it exists in the interview will get you caught

Golden rule for the interview: when you don't know or it isn't built, say: "That's not in the v1 implementation — here's exactly what v1 does instead, and here's how I would add it." That sentence, said calmly, is worth more than any bluff.

TABLE OF CONTENTS

1. Project Executive Summary (30s / 1min / 3min / 5min)
2. Complete Architecture (+6 diagrams)
3. Complete Project Structure (file → purpose → interview relevance)
4. Every Important Class Explained
5. Escrow Domain From Zero
6. State Machine (+30 questions)
7. Double-Entry Ledger (+numeric walkthroughs)
8. Database Transactions & ACID (exact boundaries)
9. Concurrency (the two-releases walkthrough)
10. Idempotency (honest audit)
11. Spring Boot Concepts → actual files
12. JPA + Hibernate → actual files
13. PostgreSQL → reverse-engineered schema
14. Security From Zero → actual request trace
15. JWT Deep Dive
16. Ownership / Authorization (honest gaps)
17. Scheduler / Expiry
18. 30+ Failure Scenarios
19. Testing (honest audit + proposed tests)
20. 20 Most Interview-Important Code Walkthroughs
21. "Why did you use X?" (15 answers)
22. Project Design Questions (10 deep answers)
23. INTERVIEW ATTACK MODE — 50 basic / 50 intermediate / 50 advanced / 30 brutal
24. Final Cheat Sheet (≤10 pages)
25. Final Rule

1. PROJECT EXECUTIVE SUMMARY

1.1 What RevertPay is ●

RevertPay is a **mini escrow-based payment platform**: a buyer sends money not directly to the seller, but into a **system-controlled holding account (ESCROW_HOLDING)**. The seller ships the item; only when the buyer **confirms delivery** does the system **release** the money from the holding account to the seller. If something goes wrong, the buyer can **raise a dispute** which freezes the funds for review. Every money movement is recorded in a **double-entry ledger** (one DEBIT + one CREDIT per transfer), and every escrow moves through a **validated state machine** with a full **audit log of status transitions**.

All of this is ● confirmed from: EscrowService, LedgerService, EscrowTransitionService, AccountService.createEscrowHoldingAccount(), EscrowStatusLogService, DisputeService.

1.2 The problem it solves ●/●

● Inference from the code's own UI copy (EscrowInitiate.jsx literally says: "Review a payment before funding... No funds move while the payment is initiated" and BuyerEscrows.jsx: "Confirm delivery to release the payment to the seller"):

Online buyers and sellers don't trust each other. A buyer fears paying and receiving nothing; a seller fears shipping and never being paid. Escrow removes the trust requirement: money is parked with a neutral third party (RevertPay's system account) until the trade completes.

1.3 Who uses it ●

Three roles, from com.revertpay.user.Role:

Role	What they do in v1 (all confirmed in code)
BUYER	Initiates escrow, confirms/cancels payment, funds it, confirms delivery (releases money), raises disputes
SELLER	Sees funded escrows, marks them SHIPPED, receives money on release
ADMIN	Exists in the enum and in the registration UI dropdown — but no admin-only endpoint or capability exists anywhere in the backend ● (do not claim an admin panel)

1.4 Why escrow exists / why reversal matters ● (domain reasoning, code-consistent)

- Escrow = "trust, engineered": the platform temporarily controls the money, so both parties cooperate.
- "Revert" (the name!) = the money must be able to come **back**: cancellation before funding, expiry of unconfirmed payments, and (as a modelled state) refunds after disputes. ● the states CANCELLED, EXPIRED, REFUNDED exist; ● note: a REFUND that actually moves money back is **not implemented** — only the state transition UNDER_REVIEW → REFUNDED is modelled.

1.5 Main actors & workflows ●

Actors: Buyer (browser), Seller (browser), RevertPay backend (Spring Boot), PostgreSQL (Neon cloud DB), the system account ESCROW_HOLDING.

Main workflows (all confirmed):

1. **Register/Login** → BCrypt password + JWT (AuthService)

2. **Auto-provision account + ₹5000 demo funding** (`AccountService.createAccount` transfers ₹5000 from `ESCROW_HOLDING`)
3. **Escrow lifecycle** → initiate → confirm → fund → ship → confirm-delivery (release) with cancel, expire (scheduler), dispute as side paths
4. **Ledger recording** → every movement = one `LedgerService.transfer` = 1 DEBIT + 1 CREDIT
5. **Manual P2P transfer endpoint** (`POST /api/auth/transfer`) — a plain double-entry wallet transfer between any two accounts ●
6. **Viewing** → `/api/me` derives your balance from the ledger ●

1.6 Main business rules ●

1. Only **BUYER** can initiate escrow; target account must belong to a **SELLER**; buyer ≠ seller.
2. A payment starts in `PAYMENT_INITIATED` and **must be confirmed within 40 seconds** (`confirmationDeadline = now + 40s`) or the scheduler expires it.
3. Money only moves via the ledger, always debit+credit pairs, never a bare balance update.
4. Escrow status changes only through `EscrowTransitionService.transition()`, which rejects illegal transitions and writes an `EscrowStatusLog` row.
5. Funding requires the caller's JWT to belong to the escrow's buyer; shipping requires the seller.
6. Release (confirm-delivery) moves money `ESCROW_HOLDING` → seller only from `SHIPPED` state.
7. Every escrow insert carries a unique `idempotencyKey` (DB unique constraint).

1.7 Your four elevator pitches (say them like a student, not a brochure)

** 30-second version**

"RevertPay is an escrow payment app I built with Spring Boot and PostgreSQL. A buyer's money goes into a system escrow account instead of straight to the seller; when the buyer confirms delivery, a double-entry ledger transaction releases it to the seller. Every escrow is a state machine — initiated, confirmed, funded, shipped, released — with illegal transitions rejected and every transition audit-logged. Auth is JWT + Spring Security, and each escrow insert is guarded by a unique idempotency key."

** 1-minute version** — add:

"The interesting part is the money safety model. There is no balance column anywhere — balances are always derived by summing an append-only ledger table, which gives me a complete audit trail for free. Each transfer writes one debit and one credit inside a single DB transaction, so totals always reconcile. Escrow state changes go through one centralized transition service protected by JPA optimistic locking (`@Version`), so two concurrent releases can't double-pay. A scheduler expires payments the buyer doesn't confirm within 40 seconds. I also know v1's honest limits: the refund money flow isn't built yet, idempotent replay of retries isn't wired, and balance checks before funding are missing — I can walk you through exactly how I'd add each."

** 3-minute version** — add the architecture sentence:

"It's a two-module repo: a React/Vite SPA and a Spring Boot 4 API. Frontend talks HTTP with a Bearer token; a filter authenticates the JWT, controllers stay thin, DTOs validate input, services hold business rules, Spring Data repositories hit PostgreSQL on Neon. The escrow flows are: initiate (validated, idempotency-keyed) → confirm/cancel within 40s → fund (buyer→escrow ledger transfer) → seller ships → buyer confirms delivery (escrow→seller transfer) or raises a dispute (`UNDER_REVIEW`). A 5-second scheduler sweeps expired initiations. Every transition is logged with actor and reason."

** 5-minute deep version** — add the engineering trade-offs:

"Design decisions I can defend: (1) **Derived balances** — appending entries is concurrency-safe (no read-modify-write on a balance row) and self-auditing; cost is $O(n)$ reads, which I'd fix with snapshot

columns or materialized aggregates. (2) **Centralized state machine** — one `isValid(current,new)` switch, so no endpoint can invent an illegal move; terminal states are CANCELLED, EXPIRED, RELEASED, REFUNDED. (3) **Optimistic locking** with `@Version` on the escrow row — two racing releases both read SHIPPED, but only one commits; the loser's ledger write rolls back with it because funding/release methods are `@Transactional`. (4) **Double-entry** — debit total always equals credit total, so money can't appear or vanish silently. Known gaps I'll volunteer before you find them: no insufficient-funds check yet, no refund flow, auto-release job is a stub, account lookup endpoints lack ownership checks, and the exception handler is still empty so errors surface as HTTP 500."

2. COMPLETE ARCHITECTURE

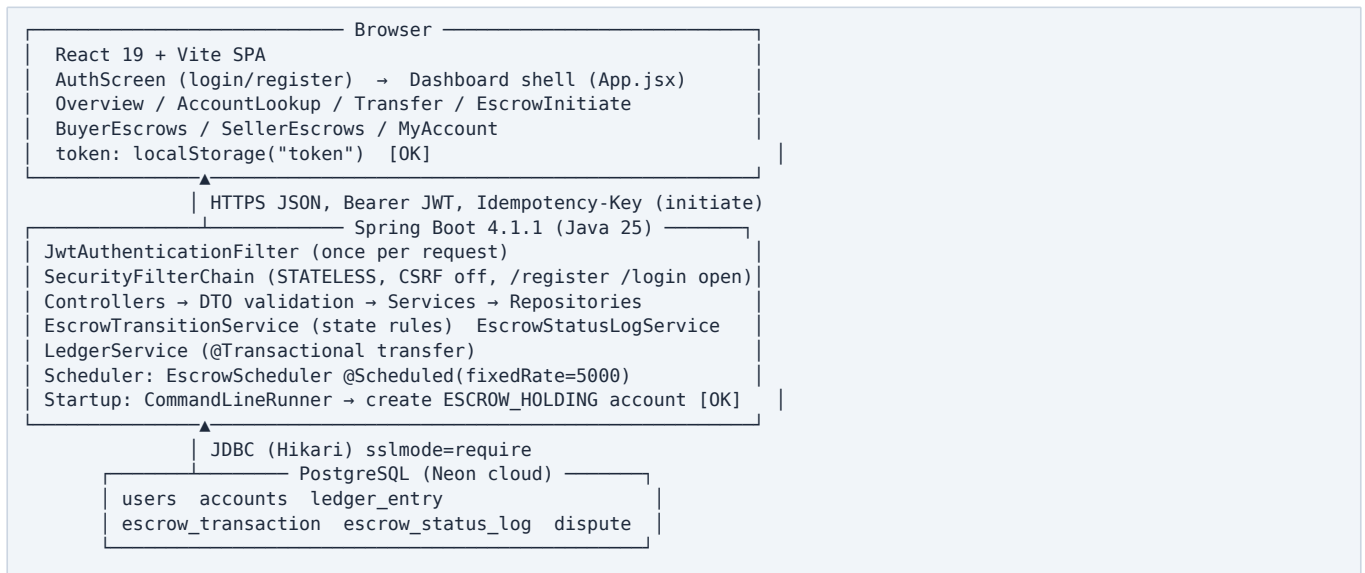
2.1 Layer stack with the ACTUAL classes ●

```

React SPA (Revertpay/frontend)
App.jsx (page state routing, token in localStorage)
components/EscrowInitiate.jsx, BuyerEscrows.jsx, SellerEscrows.jsx, Transfer.jsx, MyAccount.jsx, ...
services/api.js (endpoint registry), utils/formatters.js (readResponse, formatMoney)
    | fetch() + Authorization: Bearer <JWT> (Vite dev proxy /api → localhost:8080, vite.config.js)
    ▼
HTTP / JSON
    ▼
Controllers (@RestController) — thin: extract Bearer token, call service
user/AuthController      /api/auth/register, /api/auth/login (permitAll)
user/UserController      /api/me
account/AccountController /api/accounts/**
ledger/LedgerController  /api/auth/transfer (yes, under /api/auth/ – confirmed)
escrow/EscrowController  /api/escrow/**
dispute/DisputeController api/escrow/{id}/dispute
    ▼
DTOs + Bean Validation (com.revertpay.dto + jakarta.validation)
RegisterRequest, LoginRequest, EscrowInitiateRequest, TransferRequest, DisputeRequest
    ▼
Services (@Service) — ALL business rules live here
AuthService, AccountService, LedgerService, EscrowService,
EscrowTransitionService, EscrowStatusLogService, DisputeService, JwtService
    ▼
Repositories (Spring Data JPA interfaces)
UserRepository, AccountRepository, LedgerEntryRepository,
EscrowRepository, EscrowStatusLogRepository, DisputeRepository
    ▼
JPA / Hibernate (ddl-auto=update, show-sql=true)
    ▼
PostgreSQL @ Neon (ep-proud-scene-...neon.tech, sslmode=require) [OK] application.properties

```

2.2 Architecture diagram (components)



2.3 Request-flow diagram (POST /api/escrow/{id}/fund)

```

BuyerEscrows/EscrowInitiate.jsx
  fetch("/api/escrow/7/fund", {headers:{Authorization:"Bearer <jwt>}}) [OK]
    | Vite proxy (/api → :8080) [OK] vite.config.js
  ▼
JwtAuthenticationFilter.doFilterInternal [OK]
  header present? → jwtService.isTokenValid(token) (signature + expiry) [OK]
  → userRepository.findById(userId) → SecurityContext ← [User, ROLE_BUYER] [OK]
  ▼
SecurityFilterChain → anyRequest().authenticated() → pass [OK]
  ▼
EscrowController.fundTransaction(id, request) [OK]
  extracts raw token, calls service [OK]
  ▼
EscrowService.fundTransaction(id, token) @Transactional [OK]
  1. jwtService.isTokenValid(token) (re-check – belt & braces, also redundant) [OK]
  2. escrowRepository.findById(id) → 404-style RuntimeException if absent [OK]
  3. token userId must equal escrow.buyer.id else RuntimeException [OK]
  4. accountRepository.findByIdByAccountNumber("ACC"+userId) → buyer account [OK]
  5. accountRepository.findByIdByAccountNumber("ESCROW_HOLDING") → escrow account [OK]
  6. ledgerService.transfer(buyer, escrow, amount, escrowId) [OK]
    └─ doDebit(buyer) + doCredit(escrow) – 2 INSERTs, ONE tx [OK]
  7. transitionService.transition(tx, FUNDED, buyerId, "Funded Successfully!") [OK]
    └─ isValid(CREATED→FUNDED)? set status + save EscrowStatusLog [OK]
  8. escrowRepository.save(transaction) (version++ optimistic lock) [OK]
  ▼
EscrowInitiateResponse {escrowid, sellername, itemName, amount, status} → 200 [OK]

```

2.4 Data-flow diagram (one rupee through the system)

```

Registration:      ESCROW_HOLDING —DEBIT 5000— (system mints demo money [OK])
                  ▼
                  ACC{buyerId} = balance 5000 (derived: Σcredits-Σdebits) [OK]

Fund escrow 7 (₹600):
  ledger_entry(entry_id=101, account=ACC5, DEBIT, 600, ref=7) [OK]
  ledger_entry(entry_id=102, account=ESCROW_HOLDING, CREDIT, 600, ref=7) [OK]
  escrow.status: CREATED → FUNDED (version n → n+1) [OK]

Release escrow 7 (₹600):
  ledger_entry(entry_id=103, account=ESCROW_HOLDING, DEBIT, 600, ref=7) [OK]
  ledger_entry(entry_id=104, account=ACC9(seller), CREDIT, 600, ref=7) [OK]
  escrow.status: SHIPPED → RELEASED [OK]

Invariant: Σ(all DEBIT amounts) == Σ(all CREDIT amounts) – always [INFER]
(by construction: every ledgerService.transfer writes exactly one of each [OK])

```

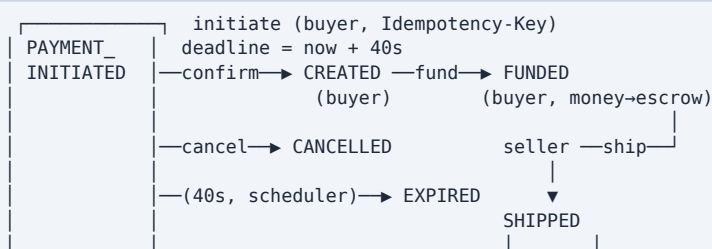
2.5 Authentication flow

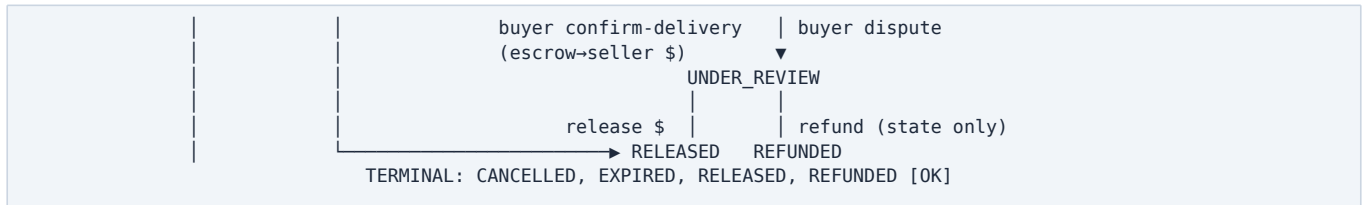
```

login: AuthController → AuthService.verify → BCrypt matches → JwtService.generateToken
      token = HS256({sub:userId, email, role, iat, exp:now+1h}, jwt.secret) [OK]
      frontend: localStorage.setItem("token", jwt) [OK] App.jsx
every request:
  Authorization: Bearer <jwt>
  → JwtAuthenticationFilter: parse+verify → load User from DB →
    UsernamePasswordAuthenticationToken(user, null, [ROLE_<role>]) → SecurityContext [OK]
  → SecurityFilterChain authorization: /api/auth/register, /api/auth/login permitAll;
    everything else .authenticated() [OK]
  → role-based checks happen INSIDE services (buyer-only / seller-only methods) [OK]
    (no @PreAuthorize / hasRole rules anywhere – confirmed absent [GAP])

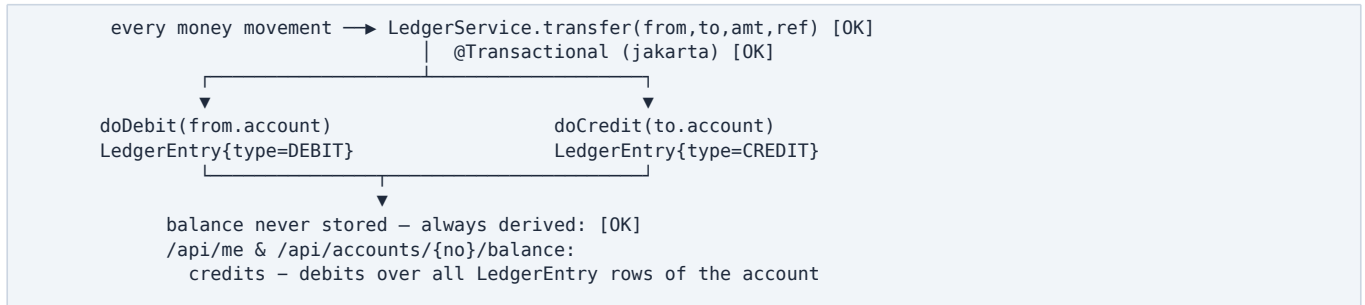
```

2.6 Escrow flow diagram





2.7 Ledger-flow diagram



3. COMPLETE PROJECT STRUCTURE

3.1 Backend — Revertpay/backend (Maven, Spring Boot 4.1.1, Java 25 ● pom.xml)

File	Purpose	Key collaborators	Interview relevance
pom.xml	Build file	—	Spring Boot 4.1.1 , Java 25 , starters: data-jpa, security, validation, webmvc ; postgresql driver (runtime), Lombok, jjwt 0.13.0 (api/impl/jackson). ●
RevertPayApplication.java	@SpringBootApplication + @EnableScheduling ●	AccountService	CommandLineRunner creates ESCROW_HOLDING at startup ●
user/			
User.java	@Entity table users: id (IDENTITY), name, email (unique, not null), password (BCrypt hash), Role enum (STRING) ●	—	Lombok @RequiredArgsConstructor → ctor(name,email,password); role set via setter ●
Role.java	enum BUYER, SELLER, ADMIN ●	—	RBAC basis
UserRepository.java	findByEmail, existsByEmail ●	—	Derived queries
AuthService.java	register (dup-email check → BCrypt → save user → create ACC{id} account → ₹5000 seed transfer) ●; verify (BCrypt match → JWT) ●	UserRepository, PasswordEncoder, JwtService, AccountService	Not @Transactional ● → partial-failure window (see §18.9)
AuthController.java	POST /api/auth/register, POST /api/auth/login, @Valid DTOs ●	AuthService	Returns raw JWT string from login ●
UserController.java	GET /api/me — user + account + derived balance + full ledger ●	UserRepository, AccountService, LedgerEntryRepository, JwtService	Balance = $\Sigma \text{credit} - \Sigma \text{debit}$ in a Java loop ●
UserService.java	EMPTY STUB ●	—	Say: "placeholder I never filled; logic lives in AuthService"
TestController.java	GET /api/test → "You are authenticated!" ●	—	Used to prove the filter works
account/			

File	Purpose	Key collaborators	Interview relevance
Account.java	@Entity accounts: id, ownerType (USER/SYSTEM), ownerId, accountNumber (unique), createdAt via @PrePersist ●	—	No balance column (commented-out remains visible in code ●) — ledger-first design
OwnerType.java	USER, SYSTEM ●	—	How ESCROW_HOLDING is distinguished
AccountRepository.java	findByAccountNumber, existsByAccountNumber ●	—	
AccountService.java	createEscrowHoldingAccount() (idempotent by exists-check) ●; createAccount() (saves + seeds ₹5000 from escrow account for USERS) ●; findByAccountNumber, saveAccount ●	AccountRepository, LedgerService	Demo-money decision — be ready to justify
AccountController.java	GET /api/accounts/{no}, /{no}/ledger, /{no}/balance, POST /new (accepts raw entity!) ●	AccountService, LedgerEntryRepository	No ownership checks — know this cold (§16)
ledger/			
LedgerEntry.java	@Entity: entry_id PK, @ManyToOne account, type (DEBIT/CREDIT STRING), amount BigDecimal, createdAt @PrePersist, reference_id ●	—	The audit trail; append-only in practice
Type.java	DEBIT, CREDIT ●	—	
LedgerType.java	EMPTY STUB ●	—	
LedgerEntryRepository.java	findByAccountId ●	—	
LedgerService.java	doDebit, doCredit, transfer = both, @Transactional (jakarta) ●	LedgerEntryRepository	THE money primitive — one debit + one credit atomically
LedgerController.java	POST /api/auth/transfer (any→any account, amount>0, different accounts) ●	AccountRepository, LedgerService	Security hole + teaching case (§16, §18)
escrow/			

File	Purpose	Key collaborators	Interview relevance
EscrowTransaction.java	@Entity: buyer/seller @ManyToOne User, itemName, amount, status, createdAt, shippedAt, autoReleaseAt, resolvedAt, resolution, idempotencyKey (unique, not null) , requestHash, @Version , confirmationDeadline ●	—	Optimistic locking + idempotency live here
EscrowStatus.java	9 states ●	—	\$6
EscrowRepository.java	findByIdempotencyKey, findByBuyerId, findBySellerId, findByStatus, findById(long) ●	—	findByIdempotencyKey is declared but never called ●
EscrowService.java	initiate / confirm / cancel / fund / ship / confirm-delivery / getTransactions / checkExpiry ●	transition, repos, ledger, jwt	The heart — \$4
EscrowController.java	7 endpoints under /api/escrow ●	EscrowService	Bearer extraction repeated in every method ●
EscrowTransitionService.java	transition() + private isValid() switch ●	EscrowStatusLogService	Centralized state machine
EscrowStatusLog.java / ...LogRepository / ...LogService	Audit rows: from, to, actorId, reason, timestamp ●	—	"Who changed what when, and why"
InvalidTransitionException.java (escrow pkg)	extends RuntimeException ●	—	Thrown by transition(); no handler maps it → HTTP 500 ●
EscrowScheduler.java	@Scheduled(fixedRate=5000) → all PAYMENT_INITIATED → checkExpiry ●	EscrowRepository, EscrowService	\$17
dispute/			
Dispute.java	@Entity: escrow FK, reason enum, description, createdAt, status (OPENED/RESPONDED/RESOLVED), resolvedAt ●	—	
DisputeReason.java	7 reasons: ITEM_NOT_RECEIVED, ITEM_NOT_AS_DESCRIBED, DAMAGED_ITEM, WRONG_ITEM, COUNTERFEIT_ITEM, SELLER_ISSUE, OTHER ●	—	

File	Purpose	Key collaborators	Interview relevance
DisputeStatus.java	OPENED, RESOLVED, RESPONDED ●	—	No transitions service for disputes — only OPENED is ever set ●
DisputeRepository.java	empty JpaRepository ●	—	
DisputeService.java	Underreview(token,id,req): token valid, escrow exists, caller must be buyer , creates Dispute(OPENED) + transitions escrow SHIPPED → UNDER_REVIEW ●	disputeRepo, jwt, escrowRepo, transition	
DisputeController.java	POST api/escrow/{id}/dispute (note: no leading slash — Spring normalizes ●)	DisputeService	
DisputeResolution/ResponseEmpties.java	EMPTY STUBS ●	—	Admin resolution flow NOT built — say so
dto/	RegisterRequest (@NotBlank @Email, role @NotNull), LoginRequest, EscrowInitiateRequest (@Positive amount; ▲ others use Lombok @NonNull, not Bean Validation ●), EscrowInitiateResponse (public fields + meaningless validation annotations ●), TransferRequest (@NotBlank accounts; amount unvalidated at DTO level ●), DisputeRequest (manual getters/setters ●)	—	Great "why DTO + validation" material
exception/	GlobalExceptionHandler, InsufficientFundsException, IdempotencyConflictException, ConcurrentModificationException, InvalidTransitionException — ALL EMPTY STUBS, none used ●		So every business RuntimeException → HTTP 500 ; validation → 400 by framework default ●
security/			
JwtService.java	HS256 key from jwt.secret prop; generateToken (sub=userId, email+role claims, 1h expiry); extractClaims/UserId; isTokenValid (verifyWith + expiry check + try/catch→false) ●	—	\$15

File	Purpose	Key collaborators	Interview relevance
JwtAuthenticationFilter.java	OncePerRequestFilter: Bearer → validate → load user from DB → authority ROLE_<role from DB> → SecurityContext ●	JwtService, UserRepository	Note: uses DB role, not token role claim ●
CustomUserDetailsService.java	EMPTY STUB ●	—	"I authenticate from the filter directly; a UserDetailsService would be the cleaner SPI"
config/			
SecurityConfig.java	PasswordEncoder = BCrypt; filter chain: CSRF off, CORS on, STATELESS , permit register/login, anyRequest().authenticated().filter before UsernamePasswordAuthenticationFilter; CorsConfigurationSource (localhost:5173) ●	JwtAuthenticationFilter	\$14
CorsConfig.java	MVC-level CORS also localhost:5173 ●	—	Two CORS configs — know why (defense for preflight)
JwtConfig.java	EMPTY STUB ●	—	
common/	Bruno/BrunoService ("Bow Bow from Bruno to Shivuuu" ●), TestyController — toy endpoints; ApiResponse, ErrorResponse — EMPTY ●	—	Delete before interview demo; own it with humor if asked
scheduler/	EscrowAutoReleaseJob — EMPTY STUB ●	—	Auto-release after autoReleaseAt NOT implemented — \$17
application.properties	Neon PostgreSQL URL + plaintext credentials , ddl-auto=update, show-sql=true, jwt.secret committed in plaintext , security logging TRACE ●	—	Biggest "what would you fix" answer

3.2 Tests — src/test/java ●

UserRepositoryTest.java and EscrowRepositoryTest.java exist but are **100% commented out** (even the package line). The configured test dependencies (spring-boot-starter-data-jpa-test, security-test, webmvc-test, validation-test) are present in the pom ● but **zero active tests ship with v1**.

3.3 Frontend — Revertpay/frontend (React 19, Vite 8 ●)

File	Purpose	Interview relevance
vite.config.js	Dev proxy /api → http://localhost:8080, changeOrigin ●	Why no CORS in dev? Proxy makes it same-origin; CORS config is the belt for direct calls
src/App.jsx	Token in localStorage; state-based page routing (page = overview/accounts/create/transfer/escrow/seller-escrows/buyer-escrows/my-account) ●	No react-router in use (AppRoutes.jsx is empty) ●
components/AuthScreen.jsx	Login/Register dual-mode form; role dropdown includes ADMIN ●	Anyone can self-register as ADMIN — real finding (\$16)
components/EscrowInitiate.jsx	initiate → inline confirm/decline → fund flow; sends Idempotency-Key: escrow-{Date.now()}-{random} freshly generated per submit ●	Core of \$10
components/BuyerEscrows.jsx	Buyer order list, confirm-delivery, dispute form (reason+description), summary stats, shows autoReleaseAt date ●	
components/SellerEscrows.jsx	Seller list + Ship button (POST /api/escrow/{id}/ship) ●	
components/Transfer.jsx	Manual ledger transfer form (POST /api/auth/transfer) ●	
components/MyAccount.jsx	GET /api/me; re-derives credit/debit sums client-side too ●	
components/AccountLookup.jsx	Any account + balance + ledger by number ●	No ownership check server-side
components/CreateAccount.jsx	⚠ Misnamed: it actually posts to API.escrowInitiate with an Idempotency-Key — a near-duplicate of EscrowInitiate ●	If asked: "legacy page, superseded by EscrowInitiate"
services/api.js	Central endpoint registry ●	
utils/formatters.js	formatMoney (Intl INR), formatDate, readResponse (JSON or text) ●	readResponse matters: backend errors are text/JSON depending on path
Empty files ●	context/AuthContext.jsx, hooks/useAuth.js, routes/AppRoutes.jsx, pages/*.jsx, utils/constants.js, LoadingSpinner, Input, Button, Dashboard.jsx (root-level components/Dashboard.jsx has content; pages/Dashboard.jsx empty), axios in package.json but all calls use fetch ●	Honesty list

3.4 Repo-level facts

- Git remote: <https://github.com/Siva2583/RevertPay-v-1.0.git> ●; single commit d5b2dd7 "Initial commit - RevertPay v1.0" ●.

- **No project README, no docker-compose, no CI config, no migration files (Flyway/Liquibase), no schema.sql** in the repo ●.
 - `.idea/` committed at repo root (minor hygiene smell).
-

4. EVERY IMPORTANT CLASS EXPLAINED

Format per class: **What / Why / Methods / Callers / Callees / In / Out / Failure / Security / Likely questions.**

4.1 LedgerService (ledger package) ●

1. **What:** The only way money moves. `doDebit(account, amount, refId)` and `doCredit(account, amount, refId)` each insert one `LedgerEntry`; `transfer(a, b, amt, ref)` calls both inside `@Transactional` (jakarta.transaction).
2. **Why:** Guarantees the double-entry invariant — a debit without its credit is impossible; and atomicity — if the credit insert fails, the debit rolls back with it (same transaction).
3. **Who calls it:** `AccountService.createAccount` (₹5000 seed), `EscrowService.fundTransaction` (buyer→escrow), `EscrowService.confirmEscrow` (escrow→seller), `LedgerController` (manual transfer).
4. **In:** two managed `Account` entities, `BigDecimal` amount, long `referenceId`. **Out:** void (rows persisted).
5. **What can fail:** DB down → exception propagates → nothing persisted (transaction rolls back). Amount ≤ 0 or negative: **NOT checked here** — only `LedgerController` checks `<= 0` ●; escrow paths don't (amount came from validated `@Positive` at initiate ●). **No balance check** — accounts can go negative ●.
6. **Security:** none inside; callers are responsible (escrow service does ownership; controller does not).
7. **Interview traps:** "Why jakarta.transaction.Transactional not Spring's?" — Here: works the same with default `REQUIRED` semantics; Spring's adds richer integration (propagation `NESTED`, read-only hints); in a codebase review I'd standardize on Spring's. "What if the two saves hit different DB nodes?" — They can't; one transaction, one connection.

4.2 EscrowService (escrow package) ● — the core

Dependencies (constructor-injected ●): `EscrowTransitionService`, `EscrowRepository`, `UserRepository`, `JwtService`, `AccountService`, `AccountRepository`, `LedgerService`.

a) transactionRequest(EscrowInitiateRequest, token, idempotencyKey) — NOT `@Transactional` ● - Resolves seller account by `sellerAccountNumber` → seller user via `account.getOwnerId()`. - Validates token (`isTokenValid`), loads buyer + seller users, enforces: seller role == `SELLER`, `buyerId != sellerId`, buyer role == `BUYER`. Each violation → `RuntimeException` ●. - Builds `EscrowTransaction`: status `PAYMENT_INITIATED`, `createdAt = now`, `confirmationDeadline = now.plusSeconds(40)` ●, stores `idempotencyKey`. - Saves, then **calls its own `checkExpiry(savedId)` once** ● (immediately returns "Valid!!" in practice — the clock just started; only a pre-set past deadline would expire it). - **Failure:** if `existsByEmail`-style duplicate check for the `idempotencyKey` existed we'd replay; instead a duplicate key violates the DB unique constraint → `DataIntegrityViolationException` → 500 ●. - **Security:** buyer identity from JWT, never from the request body ●.

b) confirmTransaction(id, token) / cancelTransaction(id, token) — NOT `@Transactional` ● - Token valid → load escrow → caller must be the escrow's buyer ("Mismatched Credentials!!") → `transitionService.transition(tx, CREATED|CANCELLED, buyerId, reason)` → save → return response DTO. - The old hardcoded status checks (if `status != PAYMENT_INITIATED` throw) are **commented out** — the transition service now owns that rule ● (nice refactoring story: state rules moved to one place). - **Failure:** transition throws `InvalidTransitionException` (e.g., confirming an `EXPIRED` escrow) → unhandled → HTTP 500 ●. - ⚠️ **Race:** confirm vs scheduler-expire — both read `PAYMENT_INITIATED`; `@Version` makes the second commit fail (`ObjectOptimisticLockingFailureException`) ●. Also: an escrow past its 40s deadline but not yet swept can still be **confirmed** (no deadline check in confirm ●) — the 5s scheduler window is the only guard.

c) fundTransaction(id, token) — `@Transactional` ● - Token valid → escrow exists → caller is buyer → resolve `ACC{buyerId}` and `ESCROW_HOLDING` → `ledgerService.transfer(buyer, escrow, amount, escrowId)` → transition `CREATED→FUNDED` → save. - Atomicity: ledger rows + escrow status + audit log commit **together** or not at all ● (single tx: transfer is `REQUIRED` so it joins). - ● **No balance check:** buyer can fund with balance 0 → account goes negative. `InsufficientFundsException` exists as an empty, unused stub. - The old `status != CREATED` check is commented out; the state machine handles it.

d) shipTransaction(id, token) — NOT `@Transactional` ● - Caller must be the **seller** ("Invalid Mathc!!" — typo in code, own it) → transition `FUNDED→SHIPPED` → `shippedAt = now`, `autoReleaseAt = now + 5 minutes` → save ●. - ●

Nothing enforces autoReleaseAt — EscrowAutoReleaseJob is empty; auto-release is display-only today.

e) confirmEscrow(id, token) (the RELEASE) — **@Transactional** ● - Caller must be buyer → resolve seller's account ACC{sellerId} + ESCROW_HOLDING → ledgerService.transfer(escrow, seller, amount, escrowId) → transition SHIPPED→RELEASED → save. - ● resolvedAt is never set (field stays null; the UI shows "Released: —" ●).
- **Concurrency:** two simultaneous releases — both read SHIPPED → both pass transition (in-memory) → **only one** wins the UPDATE ... WHERE version=? race; loser's whole tx (including its ledger rows) rolls back ● — this is the money-safety core. See §9.

f) getTransactions(token, role) ● — token → userId → findBySellerId or findByBuyerId. Ownership by query, not by post-filtering: you only ever receive your own rows.

g) checkExpiry(id) ● — load escrow; if confirmationDeadline != null && now > deadline → transition PAYMENT_INITIATED→EXPIRED with actorId=null, reason "Expired Session" → save → "Expired!!" else "Valid!!". Called by scheduler every 5s and once inline after initiate.

4.3 EscrowTransitionService ●

- transition(tx, newStatus, actorId, reason): reads current status → isValid(cur,new)? no → throw InvalidTransitionException; yes → set status + logService.logTransition(...).
- isValid = exhaustive switch on current status (§6 for the full matrix). Terminal states → false.
- **Why centralized:** every caller (buyer actions, seller actions, scheduler, disputes) is forced through the same legality check; no endpoint can hand-craft an illegal move. **Weakness:** the check is against the in-memory entity — correctness under concurrency comes from @Version on the row, not from this check (two threads can both "legally" pass; only one commit survives).

4.4 EscrowStatusLogService / EscrowStatusLog ●

- One immutable row per transition: escrow FK, fromStatus, toStatus, actorId, reason, timestamp. Nothing ever updates or deletes logs ●.
- **Failure:** in non-transactional flows (cancel/confirm/ship/checkExpiry ●), the log insert and the escrow save are **separate implicit transactions** — a crash between them leaves a log row whose state change never persisted (audit drift). In fund/confirmEscrow/dispute... wait — dispute (Underreview) is also not annotated; fund and confirmEscrow are ● — there the log is atomic with the change.

4.5 AccountService ●

- createEscrowHoldingAccount() — called from CommandLineRunner at every startup ●; checks existsByAccountNumber("ESCROW_HOLDING") first, so restarts don't duplicate (unique constraint is the backstop ●). Owner: OwnerType.SYSTEM, ownerId 0.
- createAccount(USER, id, "ACC"+id) — saves account then, for users, transfers ₹5000 from ESCROW_HOLDING (the system "mints" demo money; escrow account balance goes negative — which is fine and expected in double-entry; explain this unprompted, it's a great senior signal).
- **Not transactional** ●: user saved (in AuthService), then account save, then ledger transfer — a crash mid-way leaves a user without an account or an account without seed money (§18.9).

4.6 AuthService ●

- register: duplicate-email check → passwordEncoder.encode(BCrypt) → new User(name, email, hash) (Lombok 3-arg ctor; id excluded) → setRole → save → create ACC{id} + seed. **No @Transactional** (see above). Duplicate emails have a DB unique constraint as backstop ●, but the pre-check produces a friendlier error ●.
- verify: findByEmail → same generic error "Invalid email or password" for unknown email AND wrong password ● (good: no user enumeration) → jwtService.generateToken(user).

4.7 JwtService ● (full detail in §15)

- `@Value("${jwt.secret}") → Keys.hmacShaKeyFor(bytes) → HS256`. Token: sub = userId, claims email, role, 1-hour expiry. `isTokenValid`: parse (signature check inside `parseSignedClaims`) + explicit `expiration.after(now)`; any exception → false (logs the stack trace ● — noisy but honest).

4.8 JwtAuthenticationFilter ●

- `OncePerRequestFilter` → per request: no Bearer header → continue anonymously (public endpoints still work; protected ones fail authorization later) ●.
- Token → `isTokenValid` → `extractUserId` → `userRepository.findById` → build `UsernamePasswordAuthenticationToken(user, null, List.of(new SimpleGrantedAuthority("ROLE_" + role)))` → set into `SecurityContextHolder` ●.
- **Key talking point:** the role comes from the **database**, not the token claim — a stale/forged role claim in an old token can't grant privileges the user no longer has ● (token role claim is generated but never read for authorization ●).
- Heavily `System.out.println` instrumented ● — say "leftover debug logging; I'd switch to SLF4J at debug level".
- Loads user from DB on **every** request ● — know the trade-off: always-fresh roles vs 1 extra query per request; alternative = cache with TTL or trust token claims for statelessness.

4.9 SecurityConfig ● (full detail in §14)

- PasswordEncoder bean → `BCryptPasswordEncoder`.
- Chain: `csrf.disable()` (stateless Bearer API — no cookie sessions to hijack; know the trade-off), `cors(...)`, `SessionCreationPolicy.STATELESS`, `requestMatchers("/api/auth/register", "/api/auth/login").permitAll()`, `anyRequest().authenticated()`, `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`.
- CorsConfigurationSource bean: origin `http://localhost:5173`, all methods, headers *, credentials allowed ●.

4.10 Controllers (AuthController, UserController, AccountController, LedgerController, EscrowController, DisputeController) ●

Pattern: thin — pull Authorization header, require Bearer, strip 7 chars, pass raw token into the service; services re-validate. DTO bodies validated with `@Valid` where annotated (register, login, initiate) ●. Know these specifics: - `EscrowController.initiate` also requires header `Idempotency-Key` (missing/blank → `RuntimeException`) ●. - `LedgerController` maps to value = `"/api/auth/"` — money transfer under the auth namespace, **requires authentication** (only register/login are `permitAll`) but **no ownership** checks ●. - `AccountController.findBalance` has a subtle wart: calls `findByAccountNumber` twice and `.get()` without `isPresent()` ● — `NoSuchElementException` if missing → 500. - `UserController.getMe` derives balance with a for-loop and returns an untyped Map ● — "works, but a response DTO would be typed and documented".

4.11 Entities recap (invariants encoded in annotations) ●

- User.email unique; Account.accountNumber unique; EscrowTransaction.idempotencyKey unique & not null; EscrowTransaction.version @Version; Dispute.escrow FK not null. All monetary amounts are `BigDecimal` ● (never double — say this proactively).

4.12 DTO rules of thumb you can quote ●

- In-DTOs validate (`@NotBlank` `@Email` `@Positive` `@NotNull`); out-DTOs shape (`EscrowInitiateResponse` with `escrowid/sellername/itemName/amount/status` ●).
- Known smell to own: `EscrowInitiateRequest` uses Lombok `@NonNull` (compile-time/ctor null checks, **not** Bean Validation) on `sellerAccountNumber/itemName` ● — only amount has a real constraint (`@Positive`; note Bean Validation treats null as valid, so amount: null slips past `@Valid`, hits the Lombok `@NonNull` setter → NPE → 500 ●).

5. ESCROW DOMAIN — FROM ZERO, THEN MAPPED TO YOUR CODE

5.1 The domain vocabulary (learn to teach it)

- **Buyer** — wants to purchase; controls when money is committed and (within limits) when it's released.
- **Seller** — ships the item; gets paid only from escrow.
- **Escrow** — money held by a neutral platform while a trade completes. Neither party can unilaterally take it.
- **Funding** — moving money from buyer's wallet INTO escrow (committed, still recoverable).
- **Confirmation (payment review)** — buyer explicitly acknowledges "yes, this payment is for this seller/item/amount" before any money moves.
- **Shipping** — seller proves/declares dispatch; starts the delivery window.
- **Delivery confirmation** — buyer says "got it" → release.
- **Release** — escrow → seller. The seller's payoff.
- **Expiry** — buyer never confirmed in time → dead transaction, no money ever moved (cheap cleanup).
- **Review/Dispute** — buyer claims a problem → money frozen mid-escrow until resolved.
- **Reversal/Refund** — escrow → back to buyer. The platform's namesake ("Revert"Pay).

5.2 Mapping domain → your implementation ●

Domain concept	In RevertPay v1
Buyer/Seller identity	users rows with Role.BUYER / Role.SELLER; escrow stores @ManyToOne buyer, seller
Escrow record	escrow_transaction row (amount, item, status, timestamps)
Escrow "pot"	accounts row account_number='ESCROW_HOLDING', owner_type='SYSTEM', created at startup by CommandLineRunner
Payment review	PAYMENT_INITIATED state; confirm → CREATED; cancel → CANCELLED; 40s deadline → EXPIRED
Funding	POST /{id}/fund → ledger transfer buyer→ESCROW_HOLDING, state → FUNDED
Shipping	POST /{id}/ship (seller only) → SHIPPED, sets shippedAt, autoReleaseAt = +5 min
Delivery confirmation	POST /{id}/confirm-delivery (buyer only) → transfer escrow→seller, state → RELEASED
Auto-release on timeout	autoReleaseAt computed and shown, job NOT implemented ●
Dispute	POST /{id}/dispute (buyer only) → Dispute row + SHIPPED→UNDER_REVIEW
Refund	UNDER_REVIEW→REFUNDED transition exists; no money movement, no endpoint ●

Domain concept	In RevertPay v1
Platform treasury	ESCROW_HOLDING — debits on registration seeding make it negative; credits from funding; debits on release. In a real system it must equal the sum of in-escrow amounts — your ledger lets you prove that: $\Sigma(\text{credits} - \text{debits})$ on ESCROW_HOLDING = total currently held ●

5.3 The complete happy-path lifecycle with data ●

```

1. Register buyer → users#5, account ACC5, +₹5000 (ledger D:ESCROW C:ACC5)
2. Register seller → users#9, account ACC9, +₹5000
3. Buyer POST /api/escrow/initiate {sellerAccountNumber:"ACC9", itemName:"iPhone 15", amount:600}
   Idempotency-Key: escrow-... → escrow#7 PAYMENT_INITIATED, deadline=+40s
4. Buyer POST /api/escrow/7/confirm → CREATED
5. Buyer POST /api/escrow/7/fund → ledger D:ACC5 600 / C:ESCROW 600 (ref 7) → FUNDED
   ACC5 derived balance: 5000-600 = 4400 ; ESCROW: -5000+k +... +600 (holds 600 for this trade)
6. Seller POST /api/escrow/7/ship → SHIPPED, shippedAt=t, autoReleaseAt=t+5min
7. Buyer POST /api/escrow/7/confirm-delivery
   → ledger D:ESCROW 600 / C:ACC9 600 (ref 7) → RELEASED
   ACC9: 5000+600 = 5600 ; ESCROW back to its prior level (net 0 for trade 7)
8. Status log rows: INITIATED→CREATED→FUNDED→SHIPPED→RELEASED (actor + reason each)

```

Side paths: cancel (step 4 alternative), expire (no step 4 within 40s), dispute (replaces step 7 → UNDER_REVIEW; resolution → RELEASED or REFUNDED — resolution endpoint ● not built).

5.4 Why escrow and not just a wallet transfer? (interview favourite)

● Because payment risk is **temporal**: the buyer pays before the item arrives. A plain transfer has no undo. Escrow inserts a **hold state between commitment and settlement**, which is exactly what a FUNDED/SHIPPED gap models in the state machine. Your ledger makes the hold visible and provable rather than being a claimed number in a column.

6. STATE MACHINE

6.1 The actual transition table ● (from `EscrowTransitionService.isValid()`)

Current	Allowed next	Actor(s) in v1	Trigger endpoint / job	DB change
PAYMENT_INITIATED	CREATED	Buyer	POST /{id}/confirm	status; log(from,to,actor,reason)
PAYMENT_INITIATED	CANCELLED	Buyer	POST /{id}/cancel	status; log
PAYMENT_INITIATED	EXPIRED	System (scheduler)	@Scheduled sweep, 40s deadline	status; log(actorId=null)
CREATED	FUNDED	Buyer	POST /{id}/fund (+@Transactional)	ledger D/C pair + status + log, one tx
FUNDED	SHIPPED	Seller	POST /{id}/ship	status + shippedAt + autoReleaseAt; log
SHIPPED	RELEASED	Buyer	POST /{id}/confirm-delivery (+@Transactional)	ledger escrow→seller + status + log, one tx
SHIPPED	UNDER_REVIEW	Buyer	POST /{id}/dispute	status + new dispute row; log
UNDER_REVIEW	RELEASED	(admin — ● no endpoint)	—	modelled only
UNDER_REVIEW	REFUNDED	(admin — ● no endpoint, no money flow)	—	modelled only
CANCELLED, EXPIRED, RELEASED, REFUNDED	— (terminal)	—	—	—

Every transition also appends one `escrow_status_log` row (from, to, actorId, reason, timestamp) ●.

6.2 Why state machines are useful (say it like this)

1. **Correctness:** money movement is only legal from specific states; encoding that in one `isValid()` means an illegal move is a rejected transition, not a bug you discover in the ledger.
2. **Auditability:** transitions are discrete events with actors and reasons — a perfect fit for the log table.
3. **Concurrency anchor:** the state column + `@Version` turn "who wins a race" into a well-defined row-level question.
4. **Testability:** the entire legality surface is one pure function — the single most testable thing in the codebase (and the first test I'd write).
5. **Communication:** buyers, sellers and support all speak the same state names; the UI literally greys out impossible buttons (`BuyerEscrows.jsx` shows dispute/confirm only when `status === "SHIPPED"` ●).

6.3 Illegal transitions — what actually happens ●/●

Any call attempting e.g. FUNDED→RELEASED (skip shipping), RELEASED→UNDER_REVIEW (too late to dispute), CANCELLED→anything → `transition()` throws `InvalidTransitionException`. With the empty `GlobalExceptionHandler`, Spring's default error handling turns it into **HTTP 500** ●. The **right** mapping is 409 Conflict — say: "the state is valid, the request conflicts with the resource's current state; that's precisely 409."

Also note the guarded double-fire: POST /{id}/fund twice → second attempt hits CREATED→FUNDED illegal from FUNDED → no second ledger transfer ● — the state machine is the de facto idempotency guard for money movement (\$10).

6.4 State invariants worth memorising ●

- Ledger money for escrow #id exists **iff** status ∈ {FUNDED, SHIPPED, UNDER_REVIEW} (funded but not yet settled/released).
- From RELEASED, Σ entries with ref=id nets to zero against ESCROW_HOLDING for that trade; buyer paid exactly amount; seller received exactly amount.
- Status log rows for an escrow always form a connected chain starting at PAYMENT_INITIATED (each from equals previous to).
- version increases by 1 per committed status change.
- Terminal states never appear in a log row's from column.

6.5 30 state-machine interview questions (rapid-fire)

1. **Where is the state machine?** → EscrowTransitionService.transition()/isValid() — one service, all callers forced through it. ●
2. **Why a hand-rolled switch, not Spring Statemachine?** → 9 states, 10 edges; a dependency would outweigh the logic; exhaustive switch gives compile-time coverage of every state.
3. **What if a new state is added to the enum?** → isValid's switch over all constants stops compiling (Java exhaustive switch) — the compiler forces me to define its edges. ● (switch is over all enum constants)
4. **Which states are terminal?** → CANCELLED, EXPIRED, RELEASED, REFUNDED. ●
5. **What happens on an illegal transition?** → InvalidTransitionException → currently HTTP 500 (empty handler) → should be 409. ●/●
6. **Why is EXPIRED only reachable from PAYMENT_INITIATED?** → Because expiry only makes sense before money/commitment; past CREATED the flow has buyer intent, so it must resolve via dispute/manager paths.
7. **Why can't FUNDED go back to CREATED (unfund)?** → Money already moved in the ledger; "unfund" would need a compensating reversing transfer (REFUND path), not a status rewind — append-only ledgers never erase.
8. **Where do transitions get validated relative to the DB write?** → In-memory before setStatus and before save; the DB only enforces via version column on concurrent commit. ●/●
9. **Two threads transition the same escrow — what happens?** → Both may pass isValid on stale reads; @Version lets exactly one commit; loser gets ObjectOptimisticLockingFailureException and its tx rolls back. ●
10. **Is the state machine thread-safe?** → It has no state (stateless service) — safety comes from the DB row, not the service.
11. **Why log transitions in a separate table instead of just the status column?** → Column = current truth; log = history (who/when/why). You can't answer "who released this?" from a mutable column.
12. **Could the log and the status diverge?** → Yes in non-transactional flows (cancel/confirm/ship/checkExpiry): log insert and escrow save are separate transactions ● — fix: annotate those methods @Transactional too.
13. **What's the actor for EXPIRED?** → actorId = null (system action) ● — distinguishable in audit from user actions.
14. **Why does confirm→CREATED exist at all (why not straight to FUNDED)?** → Two distinct decisions: intent to pay this exact escrow (review gate) vs money commitment (fund). Splitting them lets a buyer decline after review with zero money movement.
15. **What prevents confirming a stale (expired-but-unswept) escrow?** → Nothing today: confirm has no deadline check ●; only the ≤5s scheduler lag bounds it. Improvement: re-check confirmationDeadline inside confirm.
16. **Which transition moves money twice if double-clicked?** → None: second attempt is an illegal transition from the new state ● — but only if the first fully committed; a mid-flight retry that races can 500

on version conflict instead — still safe, just not graceful.

17. **Why is REFUNDED reachable only from UNDER_REVIEW?** → Refunds are a dispute-resolution outcome in this model; a general "refund from FUNDED" would be a product decision needing its own edge + flow. ● (not implemented)
 18. **How would you persist the transition table instead of code?** → A state_transitions table + load-into-memory map at startup, or a rules table; code switch is simpler and compile-checked for this size.
 19. **What invariant does the log chain give you?** → Replay: from PAYMENT_INITIATED + log sequence you can reconstruct the current status — good for reconciliation/forensics.
 20. **Does the scheduler bypass the state machine?** → No — it calls the same checkExpiry → transition path. ●
 21. **Why does SHIPPED allow both RELEASED and UNDER_REVIEW?** → Buyer's two legitimate reactions to delivery: accept (release) or contest (review). It's the only branching point after money moved.
 22. **What user action moves money OUT of escrow?** → Only SHIPPED-RELEASED (confirm-delivery) in v1 ● (refund flow absent).
 23. **How do you test the state machine?** → Pure unit tests: for every (from,to) pair assert accept/reject — $9 \times 9 = 81$ cases, no Spring needed.
 24. **What's the DB default state on insert?** → Service sets PAYMENT_INITIATED explicitly in transactionRequest ● (no DB default needed; column NOT NULL).
 25. **Why store requestHash if unused?** → Planned to fingerprint the initiating request so a retried different payload with the same key could be rejected (Stripe-style 422) — ● never wired; honest answer.
 26. **Which fields change at which state?** → createdAt (initiate), status all; shippedAt+autoReleaseAt (ship); resolvedAt should change at RELEASED but ● never set; resolution at refund — ● never set.
 27. **Is UNDER_REVIEW→RELEASED safe given no admin endpoint?** → The edge is unreachable today; if someone hits it via code it would still move money correctly (same transfer as confirm-delivery) — but no REST path triggers it ●.
 28. **What if the app crashes between ledger transfer and escrow save in fund?** → Both are in one @Transactional method → both roll back. ●
 29. **And in ship (not transactional)?** → Status+timestamps+log: log insert commits first, escrow save could fail → log shows SHIPPED, row still FUNDED (drift) — known gap, fix = @Transactional. ●/●
 30. **How would you visualize/report state distribution?** → SELECT status, COUNT(*) FROM escrow_transaction GROUP BY status + escrow_status_log for funnel/ageing (e.g., stuck SHIPPED past autoReleaseAt).
-

7. DOUBLE-ENTRY LEDGER (learn to teach this cold)

7.1 From zero

- **Ledger** = an append-only list of money movements. You never write "balance = 400"; you write events: "₹600 left A, ₹600 arrived at B".
- **Account** = a bucket whose balance is whatever the events say it is. In RevertPay: accounts rows; **there is no balance column** (a commented-out balance field is still visible in Account.java — point at it and say "I removed it on purpose").
- **Debit / Credit** = the two directions of movement for an account. Convention used here: balance = $\Sigma \text{credits} - \Sigma \text{debits}$ (so CREDIT increases a normal wallet). What matters is not the naming convention — it's that **every movement has exactly two sides**.
- **Double-entry bookkeeping** = every movement records a DEBIT on one account and an equal CREDIT on another, in one atomic transaction. Therefore:

$\Sigma(\text{all debits}) \equiv \Sigma(\text{all credits})$ — always, by construction

This is the "conservation of money" law. If it ever breaks, code wrote bad rows, period — you can detect it with one query. - **Immutable transaction**: ledger rows are insert-only; corrections are new reversing entries, never edits. (Your code has no update/delete path for LedgerEntry — only save.) - **Derived balance** = compute on read: credits – debits per account (implemented in UserController.getMe and AccountController.findBalance). - **Audit trail** = every row carries who/what/when references: account, type, amount, reference_id (escrow id or 0), createdAt.

7.2 Numerical walkthrough (matches the actual code paths)

Start: Buyer ₹1000, Escrow ₹0, Seller ₹0 (as a fresh user, buyer actually starts at ₹5000 from AccountService.createAccount — we'll show both).

Step 0 — registration seeding (both users):

#	Account	Type	Amount	Running balance
1	ESCROW_HOLDING	DEBIT	5000	–5000 (system mints; negative treasury is expected)
2	ACC_buyer	CREDIT	5000	+5000
3	ESCROW_HOLDING	DEBIT	5000	–10000
4	ACC_seller	CREDIT	5000	+5000

Check: debits 10000 = credits 10000 ✓. (₹1000 example: same shape, amounts $\times 0.2$.)

Step 1 — fund escrow #7 for ₹600 (fundTransaction → transfer(buyer, escrowAcc, 600, 7)):

#	Account	Type	Amount	Running
5	ACC_buyer	DEBIT	600	4400
6	ESCROW_HOLDING	CREDIT	600	–9400 (i.e., holding ₹600 net of later mints — subtract per-trade below)

Check: 5|6 equal-and-opposite ✓. Buyer's money now in escrow.

Step 2 — release escrow #7 (confirmEscrow → transfer(escrowAcc, sellerAcc, 600, 7)):

#	Account	Type	Amount	Running
7	ESCROW_HOLDING	DEBIT	600	back to pre-trade level ✓
8	ACC_seller	CREDIT	600	5600

Check: debits 11200 = credits 11200 ✓. Per-trade view: ESCROW net for trade 7 = +600 (credit) –600 (debit) = 0 once released; while FUNDED/SHIPPED, ESCROW net = +600 = exactly the amount owed. **That's the reconciliation query** ●:

```
SELECT COALESCE(SUM(CASE WHEN type='CREDIT' THEN amount ELSE -amount END),0) AS held
FROM ledger_entry WHERE account_id = (SELECT id FROM accounts WHERE account_number='ESCROW_HOLDING');
-- held should equal SUM(amount) of escrows in FUNDED/SHIPPED/UNDER_REVIEW
```

Failure case — what if the credit insert fails? transfer is @Transactional ● → the debit rolls back → nothing happened. Money cannot be half-moved.

7.3 Mapped to code ●

Concept	Where
Account	Account entity (id + accountNumber; no balance)
Entry	LedgerEntry (account FK, Type DEBIT/CREDIT, BigDecimal amount, reference_id, createdAt)
Movement	LedgerService.transfer = doDebit + doCredit in @Transactional
Derived balance	UserController.getMe / AccountController.findBalance — loop-sum credits – debits
Audit	rows never updated; plus escrow_status_log for the non-money story
Seeding	₹5000 from ESCROW_HOLDING at registration (AccountService.createAccount)

7.4 The six ledger questions — your scripted answers

"Why double-entry?" → Because a single-sided history can't be audited or reconciled. Double-entry gives me a conservation law ($\Sigma \text{debits} = \Sigma \text{credits}$) that the DB enforces by construction, an in-escrow figure I can prove with one query, and a complete forensic trail. It's the same model real payment cores use.

"Why immutable?" → Editing history destroys auditability and invites lost updates. If something's wrong, a reversing entry preserves both the error and its correction. Append-only also makes concurrent inserts conflict-free — no row is ever read-modify-written.

"Why derive balances?" → So the balance can never disagree with the entries (no dual source of truth), and no concurrent update-lost-write problem exists. Cost: $O(n)$ summation per read. Fix when n grows: periodic snapshot column (balance_as_of), or incremental aggregate table updated in the same transaction as the entry pair.

"What if one ledger entry succeeds and the other fails?" → It can't: both inserts happen inside one DB transaction (LedgerService.transfer is @Transactional). If the credit insert throws, the debit's insert rolls back. That's atomicity doing exactly its job.

"How do you prevent money from disappearing?" → Four layers: (1) atomic debit+credit pairs, (2) conservation invariant checkable in one SQL query, (3) escrow amount held provably equals Σ

funded-unreleased escrows, (4) every row has reference_id tying money to a business event. What I do NOT yet have: a reconciliation job or DB CHECK constraints — I'd add both.

"How would you reconcile the ledger?" → Nightly job: assert $\Sigma \text{debits} = \Sigma \text{credits}$ globally; assert ESCROW net = $\Sigma(\text{active escrow amounts})$; per-account statement recompute vs any cached number; investigate gaps in entry_id sequence (signal of rolled-back transactions — normal, but monitored); and compare escrow_status_log money events against ledger rows per escrow id (reference_id).

8. DATABASE TRANSACTIONS

8.1 Theory in 90 seconds (then map it)

- **Transaction** = a group of DB operations that commits fully or not at all.
- **ACID: Atomicity** (all-or-nothing — your fund flow's ledger pair + status + log), **Consistency** (invariants hold: FKs, uniques, your conservation law), **Isolation** (concurrent txs don't see each other's intermediate states — PostgreSQL default READ COMMITTED), **Durability** (commit = survives crash, via WAL).
- **commit / rollback** — success persists; any `RuntimeException` in a Spring `@Transactional` method marks rollback.
- **Transaction boundary** = where a transaction starts/ends. In Spring: proxy intercepts a `@Transactional` method → begin → method body → commit (or rollback on `RuntimeException`). Self-invocation caveat: calling `this.transfer(...)` from inside the same class would **bypass the proxy** — in your code `LedgerService.transfer` is called **across beans** (`EscrowService` → `LedgerService`), so the proxy applies correctly 🟢. Also note `LedgerService.transfer` joining `EscrowService`'s already-open transaction = default `REQUIRED` propagation 🟡.

8.2 EXACT boundaries in RevertPay 🟢

Method	?	One transaction covers
<code>LedgerService.transfer</code>	✓ (jakarta)	debit insert + credit insert
<code>EscrowService.fundTransaction</code>	✓ (spring)	ledger pair + status + status-log + escrow save
<code>EscrowService.confirmEscrow</code> (release)	✓ (spring)	ledger pair + status + status-log + escrow save
<code>AuthService.register</code>	✗	user save / account save / seed transfer each in separate implicit txs
<code>EscrowService.transactionRequest</code> (initiate)	✗	escrow insert alone
<code>confirmTransaction</code> , <code>cancelTransaction</code>	✗	log insert, then escrow save — separate
<code>shipTransaction</code>	✗	log insert, then status/timestamps save — separate
<code>DisputeService.Underreview</code>	✗	transition+log, escrow save, dispute save — separate
<code>EscrowService.checkExpiry</code>	✗	log insert, then escrow save — separate
Scheduler loop	✗	each <code>checkExpiry</code> self-contained; failure of one escrow aborts the rest of that tick's loop (exception propagates out of <code>expireEscrows</code> 🟢)

Fastest honest summary: "The two money-moving flows are fully transactional end-to-end; several state-only flows aren't, which can strand an audit log row — my first refactor is `@Transactional` on those service methods."

8.3 Failure drills

- **"Step 3 fails" (fund: debit OK, credit fails):** rollback; buyer still has money; escrow still CREATED; client gets 500; retry is safe (state machine rejects if it actually committed).
 - **"PostgreSQL crashes mid-fund":** uncommitted tx → WAL rollback on recovery; client sees connection error; no partial rows. Retry the fund → succeeds. Durability guarantees the committed cases.
 - **"Request times out client-side but server continues":** the server finishes and commits; client thinks it failed and may retry → second attempt hits illegal transition (500) or, for initiate, a new escrow (different key — see §10). Classic distributed-systems lesson: timeout ≠ failure.
 - **"Two @Transactional nest?"** — `EscrowService.fundTransaction` (Spring) opens; `LedgerService.transfer` (jakarta, REQUIRED) joins the same physical tx. One commit at the outer boundary.
 - **"Why does initiate not need a tx?"** — it writes exactly one row; single-statement inserts are atomic anyway. (Though wrapping it would future-proof multi-row writes.)
-

9. CONCURRENCY

9.1 Concepts → where they live in RevertPay

Concept	In RevertPay v1
Race condition	Two requests reading the same escrow state before either writes (confirm vs scheduler-expire; double release)
Lost update	Impossible on balances (never stored — derived from append-only rows ●); the real risk is on the escrow status row , where it IS handled by @Version ●
Optimistic locking	@Version Long version on EscrowTransaction ● — Hibernate adds WHERE version=? to the UPDATE; 0 rows updated → ObjectOptimisticLockingFailureException → tx rollback
Pessimistic locking	● Not used anywhere (grep @Lock → none). SELECT ... FOR UPDATE absent. Honest line: "I chose optimistic because conflicts are rare and rows are single; for hot rows I'd add @Lock(PESSIMISTIC_WRITE) or SELECT FOR UPDATE."
Isolation level	PostgreSQL default READ COMMITTED ● (nothing configured) — sufficient because the version predicate re-checks the precondition at write time
DB constraints as final guard	unique(email), unique(account_number), unique(idempotency_key), FKs, NOT NULLs ● — even if the app has a race, the DB refuses duplicate keys
Idempotency	Idempotency-Key on initiate; state machine blocks replay of fund/ship/release (\$10)

9.2 THE scenario: two requests release the same escrow (walk it exactly)

Setup: escrow #7 SHIPPED, version=3, amount ₹600. Two HTTP requests arrive ~simultaneously (double-click / retry). Both call POST /api/escrow/7/confirm-delivery → confirmEscrow ●.

T1 (req A) BEGIN (Spring @Transactional) SELECT escrow7 → status=SHIPPED, ver=3 ownership ✓ transition(): SHIPPED→RELEASED INSERT ledger: D:ESCROW 600 INSERT ledger: C:ACC9 600 UPDATE escrow SET status='RELEASED', version=4 WHERE id=7 AND version=3 → 1 row updated ✓ COMMIT ✓ (money moved once)	T2 (req B) BEGIN SELECT escrow7 → status=SHIPPED, ver=3 ownership ✓ transition(): SHIPPED→RELEASED (both pass — validation is in-memory on their own snapshots; READ COMMITTED) INSERT ledger: D:ESCROW 600 INSERT ledger: C:ACC9 600 UPDATE escrow SET status='RELEASED', version=4 WHERE id=7 AND version=3 → 0 rows (A committed version=4 first) → Hibernate: StaleObjectStateException → ObjectOptimisticLockingFailureException ROLLBACK — its two ledger INSERTs die with the transaction [OK] seller paid exactly ₹600
---	--

Why the loser's ledger rows vanish: the INSERTs and the failing UPDATE share one transaction — atomicity reverses all of them. **If confirmEscrow were not @Transactional, the loser's ledger pair would persist → double-pay.** This is the single most important sentence in your interview: the @Transactional boundary and the @Version column only protect money TOGETHER.

What the loser sees: HTTP 500 (unhandled ObjectOptimisticLockingFailureException — exception/ConcurrentModificationException.java is an empty stub ●). Correct fix: catch → **409 Conflict** with

"already processed", or better, treat "already RELEASED" as success for idempotent UX.

9.3 Second scenario: fund vs scheduler-expire (state machine collision)

- Scheduler's expireEscrows loads all PAYMENT_INITIATED ●; meanwhile buyer funds that same escrow (CREATED needed — wait, expiry only targets PAYMENT_INITIATED, so the true collision is **confirm vs expire**):
- Both read PAYMENT_INITIATED. A commits CREATED (ver+1). B's UPDATE with old version hits 0 rows → exception in scheduler thread → escrow correctly stays CREATED. Money: none moved either way. ✓
- If B commits EXPIRED first: buyer's confirm throws InvalidTransitionException (CREATED was never reached; from EXPIRED everything is illegal) → 500, no money moved. ✓
- **Scheduler robustness gap** ●: one escrow throwing aborts the whole for-loop for that tick (no try/catch per item). Others get retried next 5s tick — self-healing, but a systematically-broken escrow would log an exception every tick (no dead-letter, no poison-pill quarantine).

9.4 Third scenario: two concurrent plain transfers (POST /api/auth/transfer)

Both succeed — and should: they're independent account pairs; entries are append-only; no row is read-modify-written. This is the quiet superpower of double-entry: **money correctness without locks**. The danger in that endpoint is different: it has **no ownership and no balance check** ● — any authenticated user can move money between arbitrary accounts and drive any account negative (§16, §18).

9.5 Where RevertPay would break under real concurrency (own these)

1. **Initiate double-submit with same key concurrently** → both pass service (no findByIdempotencyKey check ●) → one insert wins, other gets DataIntegrityViolationException → 500 (ugly but safe — unique constraint is the backstop ●).
 2. **Balances**: derived by full scan per read → under load, /api/me gets slow (and findByIdAccountId loads every historical row into memory ●).
 3. **No pessimistic option** if a hot escrow got contended (rare in this domain).
 4. **Scheduler** assumes single instance; two replicas = duplicate sweeps (benign thanks to versioning, but wasteful) → ShedLock/quartz-cluster answer.
-

10. IDEMPOTENCY

10.1 From zero

- **Duplicate HTTP request** — the same logical operation arriving twice: double-click, client retry after timeout, network replay, user refresh.
- **Why it matters in payments** — retries are mandatory in unreliable networks; without idempotency every retry risks double-charging.
- **Idempotency key** — client generates one unique key per logical operation and sends it on retry too; server remembers key→result and replays the stored response instead of re-executing. (Stripe pattern.)
- **Natural idempotency vs engineered idempotency** — GET is naturally idempotent; "release escrow" is state-guarded (second attempt fails loudly), which is safe but not graceful.

10.2 What RevertPay v1 actually does ● (audit!)

Piece	Status
Client sends Idempotency-Key header on initiate	✓ EscrowInitiate.jsx and (misnamed) CreateAccount.jsx: escrow-{Date.now()}-{random}
Controller rejects missing/blank key	✓ EscrowController.initiate
Key stored on escrow, unique + not null	✓ EscrowTransaction.idempotencyKey
Duplicate replay detection (lookup key → return stored escrow)	● NOT implemented — EscrowRepository.findByIdempotencyKey exists but is never called
Same-key-different-payload detection via requestHash	● field exists, never written/read
Key reused across retries of ONE click	⚠ NO — key is generated inside submit() per click; a network-timeout retry that re-runs submit() generates a fresh key → duplicate escrow. Same-key retry only exists conceptually (e.g., curl replay).
Key on fund/ship/release/dispute	● not sent by frontend, not read by backend
De-facto protection for money endpoints	✓ state machine + @Version: replay hits illegal transition / version conflict → no double money, error surfaces as 500

10.3 Scripted answers

"What happens if the same payment request arrives twice?"

"For initiate: if the retry reuses the key, the second insert violates the unique constraint and fails — safe but returns a 500 instead of the original escrow; the replay-lookup that would return a clean 200 is declared (**findByIdempotencyKey**) but I haven't wired it yet. If the frontend retries with a fresh key — which it does today, since the key is minted per submit — you get two escrows. For fund/release, a replay is blocked by the state machine: the second attempt finds the escrow already FUNDED/RELEASED and **EscrowTransitionService** throws instead of moving money twice. So: money is safe today; graceful replay is the v2 work."

"How would you make release idempotent?"

"Three options, in increasing strength: (1) rely on state machine but catch the conflict and return 200 'already released' — idempotent from the client's view; (2) accept an Idempotency-Key header on release too, store **released_key** on the escrow row with a unique constraint, check-before/insert-catch-after; (3) a

dedicated **idempotency_keys** table (key, request_hash, response snapshot, status PENDING/DONE, expires_at) — insert-first, execute, store response; concurrent duplicates block on the PK insert. For a payments company I'd implement (3) once and reuse it on every mutating endpoint."

"Why is the client key `Date.now()+random` a problem?"

"It's unique per click, but a retry-after-timeout is a new click → new key → the server can't recognize it as the same operation. The key must be created when the operation logically starts (e.g., draft id / UUID in form state) and reused until the operation reaches a terminal result."

11. SPRING BOOT — ONLY WHAT REVERTPAY NEEDS, MAPPED TO FILES

Concept	30-second definition	In RevertPay ●
Spring / Spring Boot	Inversion-of-Control framework; Boot = opinionated auto-configuration + embedded server + starters	spring-boot-starter-parent 4.1.1; starters: webmvc, data-jpa, security, validation (pom.xml)
IoC container	The framework creates and wires your objects; you don't new collaborators	Every service receives dependencies via constructor
Dependency Injection	Dependencies passed in (here: constructor injection)	EscrowService(EscrowTransitionService, EscrowRepository, UserRepository, JwtService, AccountService, AccountRepository, LedgerService) — 7 deps, single constructor → @Autowired implicit
Bean	Object the container manages	@Service, @Component, @RestController, @Configuration classes; PasswordEncoder & SecurityFilterChain @Bean methods in SecurityConfig; CommandLineRunner in the main class
@Controller/@RestController	Web endpoint; @RestController = @Controller + @ResponseBody (return value IS the body)	All 6 controllers
@Service	Business-logic bean (semantics, not magic)	AuthService, AccountService, LedgerService, EscrowService, EscrowTransitionService, EscrowStatusLogService, DisputeService, JwtService
@Repository	(implicit via Spring Data) stereotype + exception translation to DataAccessException	Not annotated manually — interfaces extend JpaRepository and get implementation at boot
DTO	Shape object decoupled from entities; validation attach point	dto package: RegisterRequest, LoginRequest, EscrowInitiateRequest/Response, TransferRequest, DisputeRequest
Entity	JPA-managed persistent class	User, Account, LedgerEntry, EscrowTransaction, EscrowStatusLog, Dispute
Bean Validation	@Valid triggers JSR-380 constraint checks; failures → MethodArgumentNotValidException → 400	AuthController & EscrowController.initiate use @Valid ● ; other endpoints don't
Exception handling	@RestControllerAdvice/@ExceptionHandler centralizes error→HTTP mapping	● GlobalExceptionHandler is an empty class — defaults apply (validation→400, everything else→500). Say: "deliberately on my list; the design slot for it exists."

Concept	30-second definition	In RevertPay ●
REST + ResponseEntity	Resources over HTTP; ResponseEntity controls status/headers explicitly	● Not used — controllers return bodies directly (200 by default) or throw; no 201/404/409 codes are set anywhere. Own it: "v1 leans on defaults; adding ResponseEntity mapping is mechanical now that exception types exist."
HTTP status codes	200 OK / 201 Created / 400 validation / 401 unauthenticated / 403 unauthorized / 404 missing / 409 conflict / 500 server	Actual behaviour: register/login/me/etc → 200; missing/blank Idempotency-Key → 500 (RuntimeException); illegal state → 500; invalid token in service → 500; unauthenticated (no JWT) on protected endpoint → 403 by Spring Security default ●; validation failures → 400 ●
Component scanning	Boot scans the @SpringBootApplication package tree	Everything lives under com.revertpay → found automatically
Profiles/externalized config	application.properties + env overrides	Only application.properties; secrets committed ● — the #1 "what would you fix" item (jwt.secret via env var, DB creds via Vault/env)

Controller → service → repository — one concrete trace to recite: EscrowController.fundTransaction (HTTP/POST /{id}/fund, extracts Bearer) → EscrowService.fundTransaction (@Transactional, rules) → LedgerService.transfer + EscrowTransitionService + repositories → PostgreSQL.

12. JPA + HIBERNATE — MAPPED TO FILES

Concept	Definition	In RevertPay ●
JPA	Jakarta persistence specification (annotations + API)	<code>jakarta.persistence.*</code> on all entities
Hibernate	The implementation behind Spring Data JPA	<code>ddl-auto=update, show-sql=true</code> (<code>application.properties</code>)
ORM	Objects ↔ rows mapping	Entities above ↔ 6 tables
Entity	<code>@Entity</code> + <code>@Id</code> ; managed lifecycle	6 entities; IDs all <code>GenerationType.IDENTITY</code> (PostgreSQL bigserial)
Repository	Spring Data interface → query derived from method name	<code>findByEmail</code> , <code>existsByEmail</code> , <code>findByAccountNumber</code> , <code>findByAccountId</code> , <code>findByIdempotencyKey</code> , <code>findByBuyerId</code> , <code>findBySellerId</code> , <code>findByStatus</code>
Persistence context	Session-level first-level cache; one identity map per tx; dirty checking flushes changes	Inside <code>@Transactional</code> fund/release: entity mutations flushed at commit; explains why <code>escrowRepository.save</code> after <code>transition()</code> is partly redundant-but-harmless
Relationships	<code>@ManyToOne</code> etc.	<code>EscrowTransaction.buyer/seller</code> (users), <code>LedgerEntry.account</code> , <code>EscrowStatusLog.escrowTransaction</code> , <code>Dispute.transaction</code> — all unidirectional <code>@ManyToOne</code> ; no <code>@OneToMany</code> collections anywhere ● (deliberately avoids loading lists into memory)
Lazy vs eager	<code>@ManyToOne</code> defaults EAGER ; <code>@OneToMany</code> defaults LAZY	⚠ So <code>findByBuyerId N+1s</code> : each escrow fetches buyer + seller lazily-on-access — actually EAGER by default for <code>ManyToOne</code> , so the list query issues <code>1 + 2N</code> selects ●. Serialization of <code>List<EscrowTransaction></code> in <code>GET /api/escrow/buyer</code> walks buyer/seller proxies (both loaded eagerly) → works; but <code>LedgerEntry.account</code> also eager → ledger reads load accounts too. Know both the diagnosis and the fix: <code>@ManyToOne(fetch=LAZY)</code> + DTO projection or <code>@EntityGraph</code> .
Cascade	Propagate entity ops to associations	● None configured — plain FKs only; deletes are not cascaded (and nothing deletes, which is right for payments)
Transactions	Spring <code>@Transactional</code> wraps repo calls	\$8 table

Concept	Definition	In RevertPay ●
JPQL / derived queries	Query from method names; JPQL for custom	● No JPQL/@Query at all — everything derived. Say: "fine at this size; the moment I need SELECT ... FOR UPDATE or a sum, I add @Query."
N+1 problem	1 list query + N association queries	Live in getTransactions + /api/me ledger (eager ManyToOne) ● — best JPA talking point you have
@Version (optimistic lock)	numeric column checked on UPDATE	EscrowTransaction.version ● — the money-safety keystone (\$9)
Lifecycle callbacks	@PrePersist etc.	Account.prePersist & LedgerEntry.prePersist set createdAt=now() ● (note: EscrowTransaction sets createdAt manually in the service instead ● — inconsistency you can explain)

Sharp JPA details to volunteer: - EscrowRepository.findById(long) re-declares findById returning the entity instead of Optional<T> — derived query shadows the base method; quirky but legal; I'd delete it and use the inherited one ●. - Enum mapping is @Enumerated(EnumType.STRING) everywhere ● — never ORDINAL (renumbering an enum would silently corrupt data). - LedgerEntry.account @ManyToOne without @JoinColumn(nullable=false) → DDL allows null account_id ● even though @NotNull (Lombok) guards the Java side; DB-level guarantee missing. - Entities are returned directly from some endpoints (GET /api/escrow/buyer returns entities incl. buyer/seller User with **password hash**) ●⚠ — Jackson serializes password field! Massive security finding: check User — it has @Getter on password, no @JsonIgnore ●. **Anyone listing escrows receives the counterpart's password hash.** Know this before they do; propose @JsonIgnore/DTO immediately. (Verify claim: User has @Getter @Setter at class level, password field no annotations beyond @NotNull @Column — yes, serialized. This is the single biggest latent security bug in v1.)

13. POSTGRESQL — REVERSE-ENGINEERED SCHEMA

● All structural facts (tables, columns, nullability, uniques, FKs) follow from entities + `ddl-auto=update` ● for exact Hibernate-generated types (nothing in the repo pins them; no migration files exist ●).

users		accounts		ledger_entry	
id	PK identity	id	PK identity	entry_id	PK identity
name	NOT NULL	owner_type	VARCHAR NOT NULL	account_id	FK→accounts.id [INFER]nullable
email	NOT NULL UNIQUE	owner_id	BIGINT NOT NULL	type	VARCHAR (DEBIT CREDIT)
password	NOT NULL (hash)	account_number	NOT NULL UNIQUE	amount	NUMERIC [INFER](default prec/scale)
role	VARCHAR NOT NULL	created_at	NOT NULL	created_at	TIMESTAMP
(ex: BUYER,SELLER,ADMIN)				reference_id	BIGINT
escrow_transaction		escrow_status_log		dispute	
id	PK identity	id	PK identity	id	PK identity
buyer_id	FK→users.id NOT NULL	escrow_id	FK NOT NULL	escrow_id	FK→escrow NOT NULL
seller_id	FK→users.id NOT NULL	from_status	NOT NULL	reason	VARCHAR NOT NULL
item_name	VARCHAR (nullable)	to_status	NOT NULL	description	NOT NULL
amount	NUMERIC NOT NULL	actor_id	BIGINT (null)	created_at	NOT NULL
status	VARCHAR NOT NULL	reason	VARCHAR	status	VARCHAR NOT NULL
created_at	TIMESTAMP NOT NULL	timestamp	NOT NULL	resolved_at	(null)
shipped_at, auto_release_at,		(OPENED/RESPONDED/RESOLVED)			
resolved_at, resolution					
confirmation_deadline					
idempotency_key	NOT NULL UNIQUE				
	← the DB-level idempotency backstop				
request_hash	(never written [OK])				
version	NOT NULL				
	← @Version optimistic lock column				

Relationships (ER summary): users 1-N escrow_transaction (as buyer AND as seller — two FKs), escrow_transaction 1-N escrow_status_log, escrow_transaction 1-N dispute, accounts 1-N ledger_entry. No joins-table, no @ManyToMany.

Why each table exists: - users — identity + role (authN + authZ source of truth; the filter re-reads it per request ●). - accounts — separates who owns money (users vs SYSTEM) from who is a person; account_number is the public-facing handle (ACC5, ESCROW_HOLDING) ●. - ledger_entry — the money history; append-only; balances derived from it ●. - escrow_transaction — the business agreement under trade (parties, item, amount, lifecycle timestamps) ●. - escrow_status_log — lifecycle forensics (who/when/why per transition) ●. - dispute — the review case attached to a shipped escrow ●.

Indexes truth-bomb ●: Hibernate creates PK + unique-constraint indexes; **PostgreSQL does not auto-index FK columns** → findByBuyerId, findBySellerId, findByAccountId, findByStatus all sequential-scan today. First performance PR: indexes on ledger_entry(account_id), escrow_transaction(buyer_id), (seller_id), (status), maybe partial index on status='PAYMENT_INITIATED' for the scheduler sweep.

Money typing: BigDecimal in Java ● → NUMERIC in PG ● — exact decimal arithmetic; never double for money. No explicit @Column(precision, scale) ● → say "I'd pin precision=19, scale=2 (or store paise as BIGINT) to prevent a 0.005 from being stored with more scale than a rupee allows."

Time handling: LocalDateTime.now() server-local ●, no TZ column type (TIMESTAMP without tz ●) — flag "would move to Instant/timestamptz + UTC" as a production fix.

Why Neon/PostgreSQL: managed cloud Postgres with a pooler endpoint, sslmode=require&channelBinding=require ● — say "I ran a real managed DB, not an in-memory H2 — schema via ddl-auto=update for dev speed; for prod I'd switch to Flyway versioned migrations so schema changes are reviewable and repeatable."

14. SECURITY — FROM ZERO, THEN THE REAL REQUEST TRACE

14.1 Concepts (the 8 you must define crisply)

Concept	Definition	In RevertPay ●
Authentication ("who are you?")	Proving identity	JWT presented per request; verified by <code>JwtService.isValid()</code> (signature + expiry); user re-loaded from DB in <code>JwtAuthenticationFilter</code>
Authorization ("may you do this?")	Permission checks after identity	Two layers: (1) URL-level — <code>anyRequest().authenticated()</code> ; (2) business-level — services compare token <code>userId</code> vs escrow buyer/seller, and check roles (BUYER/SELLER) manually
JWT	Signed token carrying identity claims	§15
BCrypt	Adaptive password hash (salt embedded, cost factor, deliberately slow)	<code>SecurityConfig.passwordEncoder()</code> → <code>BCryptPasswordEncoder</code> ; encode on register, <code>matches()</code> on login ●
RBAC	Permissions follow roles, not individuals	Role enum BUYER/SELLER/ADMIN stored on user; service-layer checks (<code>seller.getRole() != Role.SELLER</code> , <code>buyer.getRole() != Role.BUYER</code>) ●; authorities <code>ROLE_BUYER...</code> populated in the filter ● but no URL rule or @PreAuthorize ever consumes them
Roles vs authorities	Role = named group (<code>ROLE_X</code>); authority = granular permission	Filter grants exactly one authority per user: <code>ROLE_<role></code> ●
SecurityFilterChain	Ordered servlet filter chain ending in authorization decision	<code>SecurityConfig.securityFilterChain</code> : CSRF off, CORS, STATELESS, permitAll register/login, everything else authenticated, JWT filter inserted before <code>UsernamePasswordAuthenticationFilter</code> ●
Stateless authentication	No server session; every request carries proof	<code>SessionCreationPolicy.STATELESS</code> ●; token in <code>localStorage</code> on the client ●
Password hashing	Never store plaintext; hash with salt; compare via constant-time function	BCrypt hash stored in <code>users.password</code> ●; login uses <code>passwordEncoder.matches(raw, hash)</code> ●
Ownership checks	Object-level authorization (horizontal isolation)	Service-level: <code>buyerId.equals(escrowBuyerId)</code> etc. ● — but missing on account endpoints & transfer (§16)

Concept	Definition	In RevertPay ●
Endpoint protection	Which URLs are open vs locked	Only /api/auth/register, /api/auth/login open ●; everything else requires a valid JWT

14.2 The full trace (recite this) ●

```

1. React: fetch("/api/escrow/7/fund", { headers:{ Authorization: `Bearer ${localStorage.token}` }})
2. Vite dev proxy → http://localhost:8080 (same-origin in dev; CORS configured for :5173 otherwise)
3. Servlet chain → CorsFilter → JwtAuthenticationFilter.doFilterInternal (extends OncePerRequestFilter):
   authHeader null / not "Bearer ..." → continue chain unauthenticated
   else token = header.substring(7)
   jwtService.isTokenValid(token): parseSignedClaims (HMAC verify) + exp.after(now) [try/catch → false]
   userId = Long.parseLong(claims.subject)
   userRepository.findById(userId) → must exist
   SecurityContextHolder.getContext().setAuthentication(
     new UsernamePasswordAuthenticationToken(user, null, [ROLE_BUYER]))
4. SecurityFilterChain authorization:
   "/api/escrow/7/fund" not in permitAll list → anyRequest().authenticated()
   authentication exists → proceed (no authentication → 403 by default [INFER])
5. EscrowController.fundTransaction: re-reads header, strips token, delegates
6. EscrowService.fundTransaction: re-validates token (defence-in-depth/parity), loads escrow,
   asserts token userId == escrow.buyer.id ("Invalid matching!!") ← OWNERSHIP CHECK
7. Business rules + transactional money movement ($8.2)
8. 200 with EscrowInitiateResponse (or RuntimeException → 500 today)

```

Nuances that earn points: - Dual validation (filter + service) = redundancy; the service layer is the real gatekeeper because it knows business context. - Roles pulled from DB per request → a token minted before a role change can't ride old privileges ●. - CSRF disabled is correct **for this design** (no cookie auth, Authorization header only); if you ever move the token to cookies, you must re-enable CSRF. - localStorage vs cookie trade-off: immune to CSRF, exposed to XSS — in production add CSP, consider httpOnly cookie + CSRF token instead.

15. JWT DEEP DIVE

15.1 Theory

- **JWT** = `base64url(header).base64url(payload).base64url(signature)`.
- **Header** `{"alg":"HS256","typ":"JWT"}` — algorithm + type.
- **Payload (claims)**: registered claims — `sub` (subject), `iat` (issued at), `exp` (expiry) — plus custom claims.
- **Signature** = `HMAC_SHA256(base64(header) + "." + base64(payload), secret)`. Server re-computes and compares; **any payload tamper breaks the signature**.
- **Signing vs encryption** — JWTs are signed, not encrypted: anyone can read the payload (it's base64, not ciphertext). Never put secrets in claims.
- **Verification** = signature check first, then claims (`exp/issuer/audience`).
- **Bearer token** — `Authorization: Bearer <jwt>`; "bearer" = whoever holds it may use it (no proof-of-possession) → protect it like a password.
- **Expiration** — `exp` claim; your service also re-checks `expiration.after(new Date())` explicitly ●.

15.2 Exactly how RevertPay uses it ●

- **Library**: `jjwt 0.13.0 (JwtBuilder/parser)`, HMAC via `Keys.hmacShaKeyFor(secret.getBytes(UTF_8))` → HS256. Secret ≥ 32 bytes (it's 60 chars) satisfies HS256's 256-bit minimum ●.
- **Minting (`generateToken(User)`)**: `sub = user.getId().toString()`, claims `email, role (enum name)`, `iat = now, exp = now + 1 hour` ●.
- **Verification (`extractClaims`)**:
`JwtBuilder.verifyWith(secretKey).build().parseSignedClaims(token).getPayload()` — throws on bad signature/expiry; `isTokenValid` catches everything → false ●.
- **Identity**: `extractUserId` parses `sub` → the **only** identity the server trusts; role claim is informational — authorization re-reads role from the DB ●.
- **Storage (client)**: `localStorage("token")` ●, attached to every fetch.
- **No refresh tokens, no revocation/blacklist, no logout invalidation** ● — a stolen token is valid up to 1 hour; logout just deletes it client-side. Own it + fix: short-lived access token + refresh token, or DB-checked session id claim.

15.3 Token anatomy from YOUR generator (decode-ready example)

```
header = {"alg":"HS256","typ":"JWT"}
payload = {"sub":"5","email":"buyer@test.com","role":"BUYER",
           "iat":1759142400,"exp":1759146000}      ← 1h later
signature = HMACSHA256( b64(header)+"."+b64(payload), "HelloMahiruuuu...SecretKey1234567890" )
```

Interview drill: "change sub from 5 to 9 and resend" → signature verification fails (`SignatureException` inside `parseSignedClaims`) → `isTokenValid=false` → filter skips authentication → protected endpoint → 403 ●. "Read someone's token?" → yes, payload is readable base64 — that's why it carries only id/email/role, never secrets.

16. OWNERSHIP / AUTHORIZATION (horizontal privilege escalation)

16.1 The problem statement (say it first)

"Horizontal escalation = User A performing operations on User B's objects by manipulating IDs. Vertical escalation = getting more power (role). Both must be defended."

16.2 Where ownership IS enforced ●

Every escrow action loads the escrow and compares the token's userId with the party allowed to act: - fundTransaction/confirmTransaction/cancelTransaction/confirmEscrow:

token_id.equals(transaction.getBuyer().getId()) else RuntimeException ("Invalid matching!!" / "Mismatched Credentials!!" / "Invalid Credentials!!") - shipTransaction: compares with getSeller().getId() ("Invalid Mathc!!") - DisputeService.Underreview: buyer-only check - getTransactions: list **by** buyer/seller id — data is scoped in the query itself (you can't even see others' escrows)

"What if I change the escrow ID in the URL?" → I load that escrow, then the ownership compare fails unless I'm its buyer/seller → RuntimeException → 500 (should be 403; error quality, not security, is the gap).

"What if I modify the request body?" → initiate never trusts body identity (buyer from JWT; seller resolved via the account number's ownerId ●). Amount/item are trusted by design (that's the trade being agreed).

"What prevents horizontal escalation?" → the id-vs-token compare + scoped queries; but see the gaps below.

16.3 Where ownership is NOT enforced ● (volunteer before they probe)

Endpoint	Gap
GET /api/accounts/{acc_no} (+/balance, /ledger)	Any authenticated user can read ANY account's details, derived balance and full ledger → horizontal escalation (read)
POST /api/auth/transfer	Any authenticated user moves money between ANY two accounts, no balance check → arbitrary money movement — the most severe finding in v1
POST /api/accounts/new	Creates arbitrary accounts, unvalidated body
Registration role dropdown	Anyone can self-register as ADMIN ● (AuthScreen offers BUYER/SELLER/ADMIN; backend accepts any Role) — vertical escalation by self-service; fix: admin provisioning server-side only
Escrow list serialization	Returns buyer+seller User entities → password hashes leak in JSON (no @JsonIgnore) ● — biggest latent bug; fix with DTO/@JsonIgnore immediately

Your honest closing line for §16: "v1 gets the escrow object-level checks right — every escrow action verifies the token's identity against the escrow's parties, and list endpoints are scoped by query. What it does not yet have is object-level checks on the account endpoints, ownership on the manual transfer, server-controlled role assignment, and DTO-scoped serialization of users. I can rank these and fix the top three in a day: @JsonIgnore on password, transfer ownership+balance check, account endpoints restricted to self."

17. SCHEDULER / EXPIRY

17.1 What actually exists ●

- @EnableScheduling on RevertPayApplication ●.
- EscrowScheduler.expireEscrows() — @Scheduled(fixedRate = 5000): loads **all** escrows with status PAYMENT_INITIATED and calls escrowService.checkExpiry(id) for each ●.
- checkExpiry: loads escrow → if confirmationDeadline != null && now > deadline → transition(PAYMENT_INITIATED→EXPIRED, actorId=null, "Expired Session") → save → "Expired!!" ●.
- Deadline is set at initiate: confirmationDeadline = now.plusSeconds(40) ●.
- EscrowAutoReleaseJob (auto-release after the 5-minute autoReleaseAt) is an **empty stub** ● — the field exists, the UI displays it (BuyerEscrows.jsx shows "Auto release" date), but **no code enforces it**. Also shipTransaction sets autoReleaseAt = now + 5 minutes ●.

17.2 Concepts → your implementation

Concept	RevertPay facts
Scheduled jobs	Spring's @Scheduled on a single-threaded default scheduler ● (no pool config) — jobs never overlap themselves
Background execution	Runs in the JVM process; stops with the app; resumes on restart (no persistent job queue ●)
Duplicate execution	Single instance: impossible to overlap (fixedRate single thread); multiple app instances : each runs its own sweep → duplicate EXPIRE attempts — safe because @Version lets one win ●, but use ShedLock/leader election in a cluster
Concurrent user action	Confirm-vs-expire race analysed in §9.3 — version column decides; loser rolls back; money never involved at this stage
Transaction safety	checkExpiry is NOT @Transactional ● → log insert and escrow save are separate commits (drift risk) and no per-item error isolation in the loop
Restart behaviour	Nothing lost: expiration is derived from confirmationDeadline stored in the DB — a restart of hours still expires correctly on the next tick (time-based, not memory-based) ●
Idempotent-by-design	Expiring an already-EXPIRED escrow → terminal state → isValid false → exception (after a race) or correct no-op path

17.3 The three scripted answers

"What if the scheduler runs twice?" → On one instance it can't overlap (single-threaded default scheduler, fixedRate). Across replicas it would double-sweep, but each attempt re-checks state through the transition service and @Version — one EXPIRED commit wins, the other gets an optimistic-lock failure it can ignore. For production I'd still add ShedLock so only one node sweeps — wasted work and log noise otherwise.

"What if the scheduler runs while a user confirms?" → Both load PAYMENT_INITIATED. Whoever commits first wins the row (version check); the loser's update matches zero rows and throws — if the user lost, they see a 500 and the escrow is EXPIRED; if the scheduler lost, the escrow stays CREATED and expires never. Either way no money moves at this stage, so worst case is a confusing error, not financial damage. Improvement: give confirm a deadline re-check so it fails fast with 'payment window expired' (409) instead of racing the sweeper.

"What if the server crashes?" → The sweep is stateless: deadlines live in PostgreSQL, so on restart the next tick expires everything overdue in one pass. What a crash CAN leave behind (because **checkExpiry** isn't transactional): a status-log row without the status change, or half a sweep — both self-heal on the next tick. The genuinely missing piece is auto-release: **autoReleaseAt** is stored and displayed but **EscrowAutoReleaseJob** is an empty stub — the job I'd write next, reusing **checkExpiry**'s pattern with **findByStatus(SHIPPED)**.

18. FAILURE SCENARIOS (30+) — expected → actual → weakness → improvement

Format: **Expected** / **Actual v1** / **Weakness** / **Fix**. (E=expected behaviour; A=what the code really does ● unless marked ●.)

1. DB unavailable (any request) — E: clean 503. A:

JDBCConnectionException/CannotCreateTransactionException → HTTP 500 default error body. W: no mapping/retry posture, Neon pooler latency surfaces as 500s. F: @RestControllerAdvice mapping DataAccessException→503+retryable hint; health check endpoint.

2. Duplicate initiate, same key, sequential — E: replay returns original escrow. A: second insert violates unique(idempotency_key) → 500. W: declared findByIdempotencyKey never called ●. F: lookup-first; return existing escrow (200) or 409 with stored response.

3. Duplicate initiate, same key, concurrent — E: exactly one escrow. A: exactly one ✓ (constraint), loser 500. W: ugly client error. F: catch DataIntegrityViolationException → re-fetch by key → return existing.

4. Duplicate initiate, frontend retry (fresh key) — E: one escrow. A: **two escrows** (key regenerated per submit ●). W: key lifetime wrong client-side. F: generate key once per logical operation in form state; reuse across retries.

5. Double-click fund — E: single debit. A: single debit — 2nd request → transition CREATED→FUNDED illegal → 500; or optimistic-lock 500 if truly concurrent. W: error UX. F: catch → 409 "already funded" (idempotent-ish success semantics).

6. Two concurrent releases — E: exactly one payout. A: ✓ one payout (version + @Transactional rollback, §9.2). W: loser sees 500. F: 409/200-with-note; ConcurrentModificationException stub exists for this ●.

7. Invalid state action (e.g., confirm after expiry) — E: 409 Conflict. A: InvalidTransitionException → 500. W: status-code semantics + no message contract. F: exception handler → 409 with state info.

8. Unauthorized user (no token) on protected endpoint — E: 401. A: filter skips auth → anyRequest().authenticated() rejects → 403 (default) ●. W: 401 vs 403 semantics, no AuthenticationEntryPoint custom body. F: entry point returning 401 JSON.

9. Expired JWT — E: 401 "token expired". A: isValid=false → unauthenticated → 403 ●; service-level re-checks would RuntimeException→500 on some paths (fund/cancel etc. throw inside service only when a raw invalid token is supplied). W: inconsistent surfaces. F: uniform 401 + client refresh flow.

10. Malformed JWT — E: 401. A: parseSignedClaims throws → caught → false → unauthenticated → 403 ●. W: stack trace printed to stdout ● (log noise). F: log at warn with token fingerprint only.

11. Token signed with different secret — E: rejected. A: signature exception → false → 403 ✓. W: —. F: rotate secrets with kid header support later.

12. Expired escrow confirmed within scheduler lag — E: reject. A: **confirm succeeds** (no deadline check in confirm ●). W: 40s window is approximate (up to +5s). F: if now>confirmationDeadline → EXPIRED inside confirm.

13. Fund with insufficient balance — E: 422/409 decline. A: **succeeds; account goes negative** ● (no balance check anywhere; InsufficientFundsException stub unused). W: real money rule missing. F: if derivedBalance < amount → throw inside the fund tx; add DB CHECK + balance-snapshot when scale demands.

14. Manual transfer between arbitrary accounts — E: only own accounts. A: allowed for any authenticated user ●. W: ownership missing. F: force account1 to be caller's account; add role policy.

15. Manual transfer with amount ≤ 0 — E: 400. A: returns HTTP **200** with string "Enter valid Amount!!" ●. W: 200-on-error contract breaks clients. F: 400 + DTO @Positive.

16. Transfer amount null — E: 400. A: amount.compareTo NPE → 500 ●. W: DTO lacks @NotNull. F: add validation.

17. Negative amount escrow initiate — E: 400. A: @Positive on amount → 400 via @Valid ✓. W: —. F: —.

- 18. initiate missing Idempotency-Key header** — E: 400. A: RuntimeException → 500 ●. W: status. F: 400 handler.
- 19. Seller account number not found at initiate** — E: 404. A: RuntimeException "Seller account not found" → 500 ●. W: status semantics. F: map to 404/422.
- 20. Buyer==seller** — E: reject. A: buyerId.equals(sellerId) → RuntimeException → 500 ✓ logic, wrong status ●.
- 21. Non-seller target account** — E: reject. A: "Selected user is not a seller" → 500 ✓ logic ●. F: 422 with field error.
- 22. Seller ships a non-FUNDED escrow** — E: 409. A: transition rejects → 500 ✓ safe. F: 409.
- 23. Non-owner calls ship/confirm/fund** — E: 403. A: ownership compares fail → RuntimeException → 500 ✓ safe, wrong code. F: 403.
- 24. Scheduler crashes mid-list** — E: others still processed. A: exception aborts the for-loop; remaining escrows wait for next 5s tick ●. W: no per-item isolation/poison quarantine. F: try/catch per id + error metric.
- 25. Two app instances (horizontal scale)** — E: one sweep. A: both sweep; version conflict noise; still correct ●. F: ShedLock.
- 26. App killed between log insert and escrow save (ship/cancel/confirm)** — E: all-or-nothing. A: possible log-without-change (non-tx methods ●). W: audit drift. F: @Transactional on those service methods.
- 27. Registration crash after user save** — E: no user without account. A: user exists without account (register not @Transactional ●) → /api/me would 500 for them ("Account not found"). W: partial onboarding. F: wrap register in one tx (seed transfer included).
- 28. ESCROW_HOLDING missing at fund/seed time** — E: never. A: orElseThrow("No escrow!!") → 500; can't happen because CommandLineRunner creates it at startup + unique constraint ●. W: only if someone deletes the row manually. F: assert at startup (already does via create) ✓.
- 29. Concurrent registration same email** — E: one user. A: both pass existsByEmail (race), insert race → unique(email) lets one through, other gets 500 ●. W: error quality. F: catch constraint violation → friendly 409.
- 30. Clock skew (deadline math)** — E: consistent expiry. A: single JVM clock (LocalDateTime.now()), fine solo; multi-instance skew shifts 40s window per node ●. F: UTC Instant, NTP-synced hosts.
- 31. Ledger reconciliation failure (bug writes single-sided entry)** — E: impossible. A: only transfer writes pairs (atomic) ✓; no other write path exists ●. W: no detector if someone adds one later. F: nightly $\Sigma \text{debits} = \Sigma \text{credits}$ assertion job.
- 32. Frontend network timeout then user retries release** — E: idempotent success. A: state machine blocks double pay ✓ but shows error toast (500) → user confused. F: 409→"already released" toast.
- 33. JSON body wrong types (amount:"abc")** — E: 400. A: Jackson InvalidDefinitionException/HttpMessageNotReadableException → 400 by default ● ✓.
- 34. GET /api/accounts/{no} missing account** — E: 404. A: Optional empty returned as null-ish body 200 for /accounts/{no} ● (returns empty Optional → serialized as null); balance endpoint .get() → NoSuchElementException → 500 ●. W: inconsistent. F: 404 mapping.
-

19. TESTING — HONEST AUDIT

19.1 What exists ●

- UserRepositoryTest, EscrowRepositoryTest — **entirely commented out** (even package), and even when uncommented they were @SpringBootTest save-and-print exercises against the live Neon DB, not assertions. Test dependencies are configured in pom (data-jpa-test, security-test, webmvc-test, validation-test starters ●).
- **Statement you must make:** "v1 ships no active tests. I know exactly what to add, in priority order — see below." (Confidence beats pretending.)

19.2 The highest-value tests (in order)

1. **EscrowTransitionServiceTest** (pure unit, no Spring) — enumerate all 81 (from,to) pairs; assert the \$6.1 matrix; guards every money rule.
2. **LedgerServiceIntegrationTest** (@DataJpaTest + Testcontainers PostgreSQL): transfer → exactly 2 rows, debit/credit amounts equal, reference ids set; failed credit (simulate via bad amount/null account) → **zero rows** (atomicity proof).
3. **Release concurrency test:** two threads call confirmEscrow on one SHIPPED escrow → assert exactly one ledger payout pair exists and one thread got ObjectOptimisticLockingFailureException (or 409 after the fix). This is the money test.
4. **Idempotency test:** same key twice → one escrow row (today: assert DataIntegrityViolationException; after the fix: assert 200-replay).
5. **Security tests (@WebMvcTest/MockMvc + security-test):** no token → 401/403 on all protected routes; register+login open; buyer token cannot ship; seller token cannot fund; buyer-of-other-escrow cannot confirm → ownership 403.
6. **Scheduler test:** escrow with deadline in past → expired after expireEscrows(); deadline in future → untouched; confirm-vs-expire race optional.
7. **Controller slice for initiate:** missing Idempotency-Key → 400; amount ≤ 0 → 400; seller role check.
8. **Balance derivation test:** seed + fund + release → /balance math matches manual sums.

Test-infrastructure note to say out loud: "Tests must run against Testcontainers PostgreSQL, not the Neon dev DB — **ddl-auto** tests are environment-accurate and parallel-safe."

20. CODE WALKTHROUGH — THE 20 BLOCKS YOU MUST KNOW LINE-BY-LINE

(Excerpts abridged; full source is in your repo. R = risk, Q = likely question.)

W1. RevertPayApplication.createSystemAccount ●

```
@Bean CommandLineRunner createSystemAccount(AccountService s){
    return args -> s.createEscrowHoldingAccount(); }
```

Runs once after context start; createEscrowHoldingAccount checks existsByAccountNumber("ESCROW_HOLDING") before insert → restart-safe. **Why:** the escrow pot must exist before any register/fund. **R:** two instances starting simultaneously → race → unique(account_number) lets one win, other crashes on insert ●. **Q:** "Why not a SQL migration row?" → migration is the better answer; runner is fine for ddl-auto dev flow.

W2. AccountService.createAccount (seed logic) ● Saves ACC{id}, then for USER owners transfers ₹5000 from ESCROW_HOLDING. **Hidden assumption:** ESCROW_HOLDING exists (created at startup). **R:** not transactional with the caller's user-save. **Q:** "Why mint demo money?" → no payment gateway in v1; seeding makes the ledger realistic — every rupee still traceable to a system debit.

W3. LedgerService.transfer ●

```
@Transactional public void transfer(Account a1, Account a2, BigDecimal amt, long ref){
    doDebit(a1, amt, ref); doCredit(a2, amt, ref); }
```

The atomic money primitive. **R:** no amount>0 validation here (caller's job); no lock needed (append-only). **Q:** "why jakarta vs spring @Transactional" (§4.1); "what if doCredit throws" → both roll back.

W4. LedgerEntry (entity design) ● @ManyToOne Account, Type DEBIT/CREDIT (STRING), BigDecimal amount, reference_id, @PrePersist createdAt. **Why:** one row per side of a movement; immutability by convention (no update path exists). **R:** no @JoinColumn(nullable=false) ●; no precision on amount ●; reference_id is a bare long (0 = none) — a @ManyToOne to a polymorphic reference or nullable FK would be cleaner.

W5. EscrowService.transactionRequest (initiate) ● Order of operations: seller account lookup → token check → buyer/seller user load → role checks (SELLER target, BUYER caller) → buyer≠seller → build escrow (PAYMENT_INITIATED, deadline +40s, idempotencyKey) → save → self checkExpiry. **R:** idempotency lookup skipped (relies on constraint); non-transactional (single insert, acceptable). **Q:** "why fetch seller via account.getOwnerId()" → the UI gives you an account number, the domain wants a user; account is the join point.

W6. EscrowService.confirmTransaction / cancelTransaction ● Token validity → load → buyer-only → transitionService.transition(...) → save → response DTO. **Key line to quote:** the old if(status != PAYMENT_INITIATED) guards are commented out because the transition service now owns legality — a refactor story: state rules centralized, endpoints got dumber.

W7. EscrowTransitionService.isValid ● — exhaustive switch (full table §6.1). Terminal states return false. **Q:** "compile-time guarantee?" → adding an enum constant breaks compilation until you define its edges. **R:** in-memory check; concurrency safety delegated to @Version.

W8. EscrowService.fundTransaction ●

```
@Transactional
... buyer-only check → ACC{buyerId} & ESCROW_HOLDING lookup
→ ledgerService.transfer(buyer, escrow, amount, transaction.getId())
→ transition(tx, FUNDED, ...) → escrowRepository.save(tx)
```

One transaction wraps ledger + status + log. **R:** no balance check ●. **Q:** "what commits if transition throws?" → nothing; RuntimeException marks rollback including the two ledger inserts. This block is your ACID showcase.

W9. EscrowService.shipTransaction ● — seller-only; SHIPPED; shippedAt=now; autoReleaseAt=now+5min; save. **R:** not @Transactional (log/save drift); auto-release unenforced ●.

W10. EscrowService.confirmEscrow (release) ● — buyer-only; transfer escrow→seller; RELEASED; all in one tx. The §9.2 race is about this method. **R:** resolvedAt never set ●. **Q:** "why transfer BEFORE transition inside the tx?" → order within one tx doesn't affect atomicity; both commit or both roll back.

W11. EscrowScheduler.expireEscrows ● — fixedRate 5s → findByStatus(PAYMENT_INITIATED) → per-id checkExpiry. **R:** N+1 loads; loop aborts on first exception; multi-instance duplicate sweeps. **Q:** fixedRate vs fixedDelay vs cron (fixedRate = every 5s from start time, no overlap on single thread).

W12. EscrowService.checkExpiry ● — deadline null-check matters: escrows without a deadline never expire. actorId=null marks system action. **Q:** "why not expire inside initiate?" → it does call it once inline ● — belt for immediate-expiry edge cases (e.g., tests with pre-past deadlines).

W13. JwtService.generateToken/extractClaims ● — §15. Quote parseSignedClaims = verify+parse in one step. **R:** secret in properties file; no issuer/audience claims; no token id (jti) → no revocation possible.

W14. JwtAuthenticationFilter.doFilterInternal ● — §14.2 trace. **Q:** "why OncePerRequestFilter?" → guarantees single execution per request even across dispatcher forwards. "Why load user from DB?" → fresh roles; cost: 1 query/request.

W15. SecurityConfig.securityFilterChain ● — CSRF off (header-based stateless), STATELESS sessions, permitAll register/login, filter-before-UsernamePassword (so JWT auth happens before form-login machinery would). **Q:** "why disable CSRF?" → no cookies/automated browser credentials; CSRF's threat model doesn't apply to pure Bearer-header APIs.

W16. AuthService.register ● — dup-email check → BCrypt → save user → create account → seed. **R:** not transactional (§18.27); role accepted from client incl. ADMIN ●. **Q:** "why check-then-insert if a unique constraint exists?" → friendly error first; constraint is the race backstop.

W17. AuthService.verify ● — same error string for unknown email and bad password (no user enumeration ✓). **R:** no rate limiting / lockout ● → brute force possible; mention AccountLockout/limiters as the fix.

W18. UserController.getMe balance loop ●

```
for (LedgerEntry e : ledger) { if CREDIT credits=credits.add(...) else debits=... }  
balance = credits.subtract(debits);
```

Derived balance in its purest form. **R:** loads full history each call; new BigDecimal(0) vs BigDecimal.ZERO style; untyped Map response. **Q:** "balance for 1M entries?" → snapshot column/aggregate (§22).

W19. LedgerController.transfer ● — validates amount>0 + different accounts **after** resolving both accounts; **no ownership, no balance check**; returns "Successful transfer!!!" strings with 200s even on validation failures. **Q:** use it as your own security-review demo — "if I were auditing me, this is finding #1."

W20. EscrowInitiate.jsx submit + Idempotency-Key ●

```
"Idempotency-Key": `escrow-${Date.now()}-${Math.random().toString(36).slice(2)}`
```

Generated per submit → retries get fresh keys (§10). **Q:** "fix?" → mint UUID once per logical payment in component state; reuse across retry attempts; backend replay-return once wired.

21. "WHY DID YOU USE X?" — truthful, project-specific answers

- **Why Spring Boot?** Dependency injection, embedded server, and one-annotation wiring let a solo project have a real layered architecture; starters gave me security+JPA+validation pre-integrated. I also deliberately ran a very new stack — Spring Boot 4.1 on Java 25 — which forced me to read release notes rather than copy old tutorials.
 - **Why PostgreSQL?** Payments need ACID and rich constraints. Postgres gives me transactional DDL, numeric types for money, unique constraints as race backstops (idempotency key!), and it's the industry default for fintech. I ran it as a real managed Neon instance with `sslmode=require`, not an H2 toy.
 - **Why JPA / Spring Data?** The domain is entity-heavy (6 tables, 4 FKs); derived queries (`findByBuyerId`) removed boilerplate, and `@Version` gave me optimistic locking declaratively — the single feature my money-safety story rests on.
 - **Why Hibernate (ddl-auto=update)?** It came with JPA; `ddl-auto=update` was a dev-speed choice against a cloud DB. For production I'd flip to Flyway: schema changes must be reviewable/reversible artifacts.
 - **Why React (frontend)?** I needed a stateful dashboard (auth token + multiple money views) fast; component state + fetch was enough, no router needed. If it grows, React Router/TS are drop-in.
 - **Why JWT?** Stateless horizontal scaling (no session affinity), self-contained identity for the filter, 1h expiry enforced both by the library and my explicit check. Trade-offs I accepted and know: no revocation, XSS exposure in localStorage.
 - **Why BCrypt?** Purpose-built password hashing: per-hash salt, tunable cost factor, slow by design to cap brute-force; stored in the hash string itself so no salt table. (vs SHA-256 = wrong tool: too fast, needs manual salt.)
 - **Why RBAC via roles on the user + service checks?** Three roles, a handful of rules — a full authority matrix would be ceremony. I still mint `ROLE_X` authorities in the filter so `@PreAuthorize("hasRole('SELLER')")` is a one-line upgrade path.
 - **Why DTOs?** Contract control and validation attachment (`RegisterRequest @Email @NotBlank`), decoupling JSON shape from entities, and not leaking internals — though I'll admit v1 still returns raw entities on escrow lists, which is exactly the leak DTOs prevent.
 - **Why a service layer?** Controllers stay transport-only; business rules (buyer-only, state transitions, ledger moves) live in testable beans; `@Transactional` boundaries belong there. EscrowService is proof: 7 collaborators, one responsibility — orchestrate escrow rules.
 - **Why a repository layer?** Query-by-method-name keeps SQL intent in the interface; it's where I'd add `@Lock/@Query` later without touching services.
 - **Why transactions (specifically there)?** Because "debit without credit" or "paid but not marked RELEASED" are exactly the failure classes payments must make impossible. My two money methods are `@Transactional` for that reason — and I can name the methods that aren't, and why they should be.
 - **Why a ledger (not a balance column)?** Auditability is a payments requirement, not a nice-to-have: append-only entries give replayability, reconciliation ($\Sigma D = \Sigma C$), per-escrow forensics via `reference_id`, and no lost-update problem. The cost is read-time summation, which has standard fixes.
 - **Why a state machine (hand-rolled)?** Ten edges across nine states — a switch is exhaustively checked by the compiler and readable in one screen; Spring Statemachine's XML/config weight isn't justified. Centralization means no endpoint invents an illegal move.
 - **Why a scheduler?** Deadlines need an enforcer that doesn't depend on user action: unconfirmed payments must die to keep "open" state honest. The DB stores the deadline, so restarts and crashes don't lose it. (And I'll admit the auto-release half is a stub — the expiry half is live.)
-

22. PROJECT DESIGN QUESTIONS (deep answers)

"What would you change first?" → A 5-PR sprint: (1) `GlobalExceptionHandler` → 400/401/403/404/409 mapping for the `RuntimeException` classes I already throw; (2) `@JsonIgnore/DTO` on user serialization (password-hash leak); (3) ownership + balance check on transfer & account endpoints; (4) `@Transactional` on the non-tx escrow methods; (5) wire `findByIdempotencyKey` replay + fix client key lifetime. All are small diffs with big honesty dividend.

"How would you scale it?" → Stateless API (already: JWT, no sessions) behind a load balancer; read replicas for list endpoints; move balance derivation to snapshot columns updated in-tx (or Redis cache with invalidation on ledger write); partition `ledger_entry` by time; move scheduler to `ShedLock` or a dedicated job runner; add Flyway + blue-green deploys. App tier scales horizontally today; the DB is the bottleneck by design choices (full-scan balances, N+1 lists).

"How would you handle 100,000 escrows?" → Indexes first (`buyer_id`, `seller_id`, `status`, partial on `PAYMENT_INITIATED`); paginate list endpoints (`Pageable` — trivially supported by the repositories I already have); scheduler sweep by index with `LIMIT n` batches; balances via incremental aggregates; the state machine and ledger logic do not change — good sign of a healthy core.

"How would you prevent duplicate payments?" → Three layers, matching failure modes: (1) client: stable idempotency key per logical operation; (2) API: key→response store with unique PK insert (block concurrent duplicates), request-hash mismatch → 422; (3) domain: state machine + `@Version` as the last line (already live). Plus anomaly monitoring (two funds for one escrow = page someone).

"How would you audit transactions?" → Already: append-only ledger + status log with actor/reason. Add: (a) reconciliation job ($\Sigma D = \Sigma C$, escrow-held vs ledger-held, \$7.2 query); (b) append-only enforcement in DB (`REVOKE UPDATE/DELETE` on `ledger_entry`); (c) hash-chaining (each row stores prev-row hash) for tamper evidence; (d) export to cold storage.

"How would you recover from database failure?" → Prevention: managed HA Postgres, PITR backups, async replica. Recovery drill: promote replica → app reconnects via pool (Hikari retries) → in-flight txs were lost atomically (clients retry — state machine makes retries safe) → run reconciliation before unfreezing money ops. The design principle: the DB is the only source of truth; the app holds none — that's why recovery is possible.

"How would you add refunds?" → (1) Endpoint `POST /escrow/{id}/refund` (admin/support role); (2) guard: status `UNDER_REVIEW` (edge exists in the machine already); (3) money: `ledgerService.transfer(escrowHolding, buyerAccount, amount, escrowId)` — a true reversing entry, ref same escrow id so forensics see fund+refund pair; (4) transition to `REFUNDED` in the same tx; (5) set `resolution/resolvedAt` (fields already exist, currently unused 🟢); (6) dispute row status → `RESOLVED`; (7) tests: happy path + double-refund blocked by terminal state + concurrent refund/release blocked by version.

"How would you add notifications?" → Domain events out of the transition service (it already sees every lifecycle change): publish `EscrowTransitioned(escrowId, from, to, actor)` to a queue (or `@TransactionalEventListener(AFTER_COMMIT)` + outbox table for reliability); consumers email/push buyer on `FUNDED/SHIPPED`, seller on `RELEASED`, both on `UNDER_REVIEW`. Outbox pattern keeps DB and queue consistent — no notifications for rolled-back transitions.

"How would you add payment gateway integration?" → Replace "fund = internal ledger move" with "fund = gateway authorization": create gateway `PaymentIntent` → return client secret → confirm webhook arrives → verify webhook signature → idempotent webhook handler (gateway event id as idempotency key — the pattern v1 already models) → THEN write the internal ledger pair (money in = gateway settlement account debit, user credit) → mark `FUNDED`. Gateway is just another SYSTEM-owned account in my ledger — double-entry absorbs it without redesign.

"How would you secure production deployment?" → Secrets out of git (env/Vault; rotate the committed DB+JWT secrets — assume compromised); HTTPS only; security logging off TRACE; `ddl-auto=validate` + Flyway; dependency scanning (OWASP); rate limiting on auth; CORS locked to the real domain; least-privilege DB user (app can't drop tables); the \$16 fixes; audit alerting; healthchecks; containerized image with non-root user; and the testing stack from §19 wired into CI.

23. INTERVIEW ATTACK MODE — 180 QUESTIONS

Format: **Q** → **A** (ideal answer) → **F** (follow-up) → **T** (trap to avoid).

23.1 BASIC (50)

1. **What is RevertPay?** → Escrow payment app: money held in a system account until delivery is confirmed; built with Spring Boot + PostgreSQL + React. F: "What problem?" → buyer-seller trust. T: Don't say "like PayPal" without the escrow caveat.
2. **Who are the actors?** → Buyer, Seller, Admin (role enum), platform system account. F: "What can admin do?" → nothing yet — no admin endpoints ●. T: Claiming an admin panel.
3. **What stack?** → Spring Boot 4.1.1/Java 25, Spring Data JPA, Spring Security, jjwt, PostgreSQL (Neon), React 19 + Vite. F: "Why Boot 4?" → current version; new-starters naming (webmvc starter). T: Saying Spring Boot 2 conventions (javax vs jakarta).
4. **Walk me through a payment.** → initiate→confirm→fund→ship→confirm-delivery; money: buyer→ESCROW_HOLDING→seller. F: "Where's the money between fund and release?" → ESCROW_HOLDING. T: Forgetting no money moves at initiate.
5. **What is escrow?** → Neutral third party holds funds until trade conditions met. F: "Why needed?" → removes counterparty trust. T: Vague "secure payment" hand-waving.
6. **What's the escrow account called?** → ESCROW_HOLDING, OwnerType.SYSTEM, ownerId 0, created at startup by CommandLineRunner. F: "Duplicate on restart?" → exists-check + unique constraint. T: Saying it's per-transaction (it's one global pot).
7. **How is a user's balance stored?** → It isn't — derived from ledger (credits–debits). F: "Why?" → audit + no lost updates. T: "In the accounts table" (the field is commented out on purpose).
8. **What is the ledger table?** → ledger_entry: account FK, DEBIT/CREDIT, BigDecimal amount, reference_id, createdAt. F: "Why reference_id?" → tie money to escrow. T: Calling rows "transactions" — movement is the pair.
9. **What does funding do in the ledger?** → Debit buyer, credit ESCROW_HOLDING, same amount, ref=escrowId. F: "Release?" → reversed pair. T: Forgetting both rows carry the same reference.
10. **Which states exist?** → PAYMENT_INITIATED, CREATED, FUNDED, SHIPPED, UNDER_REVIEW, RELEASED, CANCELLED, EXPIRED, REFUNDED. F: "Terminal?" → last four. T: Inventing a REFUND_IN_PROGRESS.
11. **What happens on confirm?** → PAYMENT_INITIATED→CREATED by the buyer; no money. F: "Cancel?" → CANCELLED. T: Mixing confirm-payment vs confirm-delivery.
12. **What does the scheduler do?** → every 5s, expires PAYMENT_INITIATED past its 40s deadline. F: "Who sets the deadline?" → initiate: now+40s. T: Claiming auto-release is scheduled (stub ●).
13. **How is auth done?** → JWT (HS256, 1h, sub=userId) via Authorization header; filter validates, loads user, sets ROLE authority. F: "Session?" → STATELESS. T: "Cookies" — no.
14. **How are passwords stored?** → BCrypt hash; matches() on login. F: "Why BCrypt?" → salted, slow, tunable. T: "Encrypted" — hashed, not encrypted.
15. **How do you check the caller is the buyer?** → compare jwtService.extractUserId(token) with escrow.getBuyer().getId() in the service. F: "Ship?" → seller compare. T: Saying Spring annotations do it.
16. **What's a DTO here?** → RegisterRequest/LoginRequest/EscrowInitiateRequest etc. — request/response shapes with validation. F: "Why not entities?" → contract decoupling + not leaking fields. T: Ignoring that some endpoints DO return entities (know your own gaps).
17. **Where's validation?** → @Valid on register/login/initiate DTOs (NotBlank/Email/Positive). F: "On transfer?" → manual check amount>0 in controller (+ gaps). T: Claiming all endpoints validate.
18. **What HTTP codes does the API return?** → 200s for success, 400s for validation, 403 unauthenticated (default), 500 for business exceptions (no handler yet). F: "Ideal?" → 401/403/404/409 mapping. T: Pretending polished codes exist.
19. **What is an entity? (in this repo)** → JPA-managed class mapped to a table: User, Account, LedgerEntry, EscrowTransaction, EscrowStatusLog, Dispute. F: "ID generation?" → IDENTITY all. T: Listing empty stub classes as entities.

20. **What tables exist?** → users, accounts, ledger_entry, escrow_transaction, escrow_status_log, dispute.
F: "Created how?" → Hibernate ddl-auto=update. T: Naming a "dispute_resolution" table (class is empty stub).
21. **What is @Transactional doing in fund?** → wraps ledger pair + status + log in one DB tx; rollback on exception. F: "If credit fails?" → debit rolls back. T: Thinking it retries.
22. **What is optimistic locking?** → @Version column; UPDATE with WHERE version=old; 0 rows → conflict exception. F: "Where?" → EscrowTransaction. T: Confusing with pessimistic (row locks).
23. **What is an idempotency key?** → client-generated unique key per logical operation; server dedupes. F: "Where used?" → initiate only; stored unique on escrow. T: Claiming full replay support (not wired).
24. **What is double-entry?** → every movement = equal debit+credit pair; $\Sigma D = \Sigma C$ always. F: "Prove it?" → one SQL sum query. T: "Debit means decrease" — direction depends on convention; the equality is the point.
25. **What does /api/me return?** → user info, account info, derived balance, full ledger. F: "Who can call it?" → any authenticated user, own data only (id from token). T: Saying it takes a userId param (it doesn't).
26. **What's in application.properties?** → datasource (Neon URL+creds), ddl-auto=update, show-sql, jwt.secret, security logging. F: "Problem?" → secrets committed — top fix. T: Hiding it; volunteer it.
27. **What's the frontend architecture?** → App.jsx state-based page switching; token in localStorage; components per feature; fetch via Vite proxy. F: "Router?" → not used (AppRoutes empty). T: Claiming React Router/Redux.
28. **What does the Vite proxy do?** → dev-time /api → localhost:8080, so browser sees same-origin. F: "CORS then?" → still configured for direct access (5173). T: Saying CORS is irrelevant everywhere.
29. **What happens when registration finishes?** → user row + ACC{id} account + ₹5000 seed transfer from ESCROW_HOLDING. F: "Why seed?" → no gateway in v1; demo money. T: Forgetting the escrow account goes negative as a result.
30. **What is EnumType.STRING and why?** → enum stored as its name; survives enum reordering. F: "Alternative?" → ORDINAL — dangerous. T: Not knowing the default is ORDINAL if you forget the annotation.
31. **What's @PrePersist used for?** → Account.createdAt and LedgerEntry.createdAt set at insert time. F: "Alternative?" → DB DEFAULT now() or @CreationTimestamp. T: Saying all entities use it (EscrowTransaction sets createdAt manually).
32. **What derived queries exist?** → findByEmail, existsByEmail, findByAccountNumber, existsByAccountNumber, findByAccountId, findByIdempotencyKey, findByBuyerId, findBySellerId, findByStatus. F: "SQL?" → Spring derives from names. T: Claiming custom JPQL exists (none).
33. **What does findByIdempotencyKey do today?** → nothing — declared, never called (honest gap). F: "Intended?" → replay lookup. T: Pretending it's active.
34. **What is @Version's column type/behaviour?** → Long, starts 0, +1 per update; Hibernate enforces. F: "Where visible?" → escrow_transaction.version. T: Confusing with entity id.
35. **What's a terminal state?** → no outgoing edges: CANCELLED, EXPIRED, RELEASED, REFUNDED. F: "What happens if attempted?" → InvalidTransitionException. T: Calling SHIPPED terminal.
36. **What does the filter do with no Authorization header?** → continues unauthenticated; protected endpoints later rejected. F: "Why not 401 immediately?" → public endpoints must pass. T: Saying the filter blocks the request.
37. **What's the login response?** → the raw JWT string. F: "Ideal?" → JSON with expiry + user summary. T: Claiming refresh tokens exist.
38. **What happens if you register twice with the same email?** → pre-check "Email already registered" RuntimeException → 500; DB unique constraint is backstop. F: "Concurrent?" → constraint decides, loser 500. T: Saying it returns a clean 409 today.
39. **What is OWNER-type SYSTEM account for?** → distinguishes platform's escrow pot from user wallets. F: "Could there be others?" → yes: gateway settlement, fee accounts. T: Thinking ESCROW_HOLDING is per user.
40. **What's in an EscrowStatusLog row?** → escrow FK, from, to, actorId, reason, timestamp. F: "Updated?" → never; append-only. T: Saying it stores amounts.
41. **What does ship do besides status?** → sets shippedAt and autoReleaseAt=+5min. F: "Who enforces autoReleaseAt?" → nobody yet 🚫. T: Claiming the job exists.

42. **What does confirm-delivery do?** → buyer-only; transfer ESCROW_HOLDING→seller; status RELEASED; @Transactional. F: "Idempotent?" → double-call blocked by state machine (error surface needs polish). T: Saying money moves twice.
43. **What is the response of initiate?** → EscrowInitiateResponse: escrowid, sellername, itemName, amount, status. F: "Validation on response?" → annotations present but meaningless there (design smell, own it). T: Claiming pagination.
44. **What's the dispute flow?** → buyer (on SHIPPED) → POST /{id}/dispute {reason, description} → Dispute row OPENED + escrow UNDER_REVIEW. F: "Resolve?" → no endpoint yet 🚫. T: Claiming admin resolution UI.
45. **What reasons exist for disputes?** → 7 enum values (ITEM_NOT_RECEIVED ... OTHER). F: "Validation?" → DisputeRequest @NotNull/@NotBlank. T: Inventing reasons not in the enum.
46. **What's @EnableScheduling for?** → activates @Scheduled processing. F: "Without it?" → scheduler silently never runs. T: Saying fixedRate guarantees catch-up after downtime (it doesn't skip/queue while app is down).
47. **What's IDENTITY ID generation?** → DB auto-increment (bigserial); id known after insert. F: "Alternative?" → SEQUENCE/UUID. T: Claiming UUIDs are used.
48. **What does show-sql=true do and why is it dev-only?** → logs SQL; noise + perf in prod. F: "Better?" → p6spy/logging with bindings in dev only. T: Shipping it on.
49. **What happens when an exception escapes a controller?** → today: Spring default → 500 JSON (whitelabel-ish error body). F: "Fix?" → the empty GlobalExceptionHandler slot + @RestControllerAdvice. T: Saying GlobalExceptionHandler handles it (it's an empty class).
50. **What's your favourite weakness in v1?** → pick 2: transfer endpoint lacks ownership+balance check; password-hash serialization; then give the one-day fix plan (\$22.1). T: "There are none."

23.2 INTERMEDIATE (50)

51. **Why is the balance derived instead of stored — full trade-offs?** → correctness/audit vs O(n) read; fixes: snapshot/aggregate. F: "When does it break down?" → high-frequency reads or long histories. T: Presenting derived as free.
52. **Walk the optimistic-lock release race.** → §9.2 verbatim. F: "Why does the loser's ledger vanish?" → same tx rollback. T: Forgetting @Transactional is what links ledger rollback to version failure.
53. **Which methods are transactional and which aren't — and does it matter?** → fund/confirmEscrow/transfer yes; register/initiate/confirm/cancel/ship/dispute/checkExpiry no; matters for log/save drift and register partials. F: "Quick fix?" → annotate the state-only methods. T: Saying "everything is transactional."
54. **Explain confirmationDeadline's lifecycle.** → set at initiate (+40s), read by checkExpiry (scheduler + inline), never re-checked at confirm (gap). F: "Fix?" → check inside confirm. T: Claiming confirm validates it.
55. **Why does the filter load the user from DB instead of trusting the token role?** → fresh authz, revocation-ish; cost 1 query. F: "Scale?" → cache/TTL or claims if acceptable. T: Not knowing your own code does DB-per-request.
56. **What exactly does isTokenValid verify?** → signature (parseSignedClaims throws) + explicit exp check; catch-all false. F: "Missing?" → issuer/audience/jti. T: Saying it checks user existence (that's the filter's DB load).
57. **How does initiate resolve the seller from an account number?** → accountService.findByAccountNumber → ownerId → user; then role==SELLER. F: "What if the account belongs to a buyer?" → rejected. T: Saying the DTO carries sellerId.
58. **Why is CSRF disabled — defend it.** → Bearer header, no ambient cookie credentials; token not auto-attached. F: "If you switch to cookies?" → re-enable + SameSite. T: "CSRF is legacy" — it's design-dependent.
59. **Where could an audit log row diverge from escrow state?** → non-tx methods (log commits, save fails). F: "Detect?" → reconciliation of log chain vs current status. T: Claiming it's impossible.
60. **What happens on POST /{id}/fund twice (sequential)?** → second → InvalidTransitionException → 500. F: "Better?" → 409/idempotent success. T: Saying "idempotent by design" without the error nuance.

61. **How would a duplicate initiate look at the DB level?** → second insert violates escrow_transaction_idempotency_key_key → DataIntegrityViolationException. F: "Handle?" → fetch-by-key return existing. T: Not knowing the constraint name/behaviour.
62. **Explain the N+1 in escrow listing.** → list query + eager buyer + eager seller per row (ManyToOne default EAGER). F: "Fix?" → fetch LAZY + @EntityGraph/DTO projection. T: Saying "I use lazy everywhere" (ManyToOne defaults eager).
63. **Why BigDecimal for money — and what's still missing?** → exact decimal, no float drift; missing explicit scale/precision. F: "0.1+0.2?" → BigDecimal exact vs double 0.30000000000000004. T: Using double anywhere.
64. **How does bcrypt.matches() stay safe against timing attacks?** → constant-time comparison inside BCrypt; hashes carry salt. F: "Why not SHA256+salt?" → too fast for passwords. T: Implementing your own crypto.
65. **What's the point of STATELESS session policy?** → no JSESSIONID created/used; scales horizontally; forces per-request credentials. F: "Anything still created?" → no; security context is per-request, thread-local. T: Saying HttpSession still works.
66. **Trace POST /api/auth/transfer security gaps.** → authenticated only; account1/account2 any; no balance check; 200 on validation failures. F: "Fix list?" → own-account enforcement, balance check, 400s, move out of /api/auth. T: Defending it as fine because "authenticated".
67. **Why seed via ESCROW_HOLDING debit (making it negative) rather than minting a CREDIT-only entry?** → CREDIT-only breaks $\Sigma D = \Sigma C$; debiting the treasury keeps conservation and models the platform's liability. F: "Real money equivalent?" → platform float/treasury. T: "Negative balance is a bug."
68. **How would you make the scheduler cluster-safe?** → ShedLock DB lock / Quartz clustered / leader election; locks are advisory but idempotent sweeps are the real safety. F: "Why not just @Version?" → prevents damage, not wasted duplicate work/log noise. T: Assuming Spring Scheduler coordinates across JVMs.
69. **What breaks if two app instances run?** → double sweeps (safe-but-noisy), startup race on ESCROW_HOLDING creation (unique constraint), CORS/dev unaffected. F: "Needed for scale-out?" → statelessness already OK; fix jobs + startup. T: "Sessions break" — there are none.
70. **How does the state machine + version column divide responsibilities?** → machine = business legality (single-threaded view); version = concurrency legality (commit-time truth). F: "Can both pass yet fail?" → yes, that's exactly the race. T: Treating isValid as concurrency-safe.
71. **Why 40 seconds for confirmation?** → product demo choice to make expiry observable (5s sweep, 40s deadline); real products use minutes/hours. F: "How would you configure per-marketplace?" → property-driven deadline per tenant. T: Pretending deep research went into 40s.
72. **What would requestHash have been for?** → same key + different payload → reject 422 (Stripe semantics). F: "How computed?" → canonical JSON/SHA-256 of body. T: Claiming it's active.
73. **Explain your exception hierarchy today.** → RuntimeExceptions with messages + InvalidTransitionException; handler empty → 500s. F: "Design the hierarchy?" → DomainException base → NotFound/Forbidden/Conflict/Validation subclasses → advice maps each. T: "I have a GlobalExceptionHandler."
74. **What does GET /api/accounts/{no} leak?** → any account's existence, derived balance, full ledger to any authenticated user. F: "Fix?" → restrict to owner (+admin), 404 camouflage. T: Not knowing your endpoints do this.
75. **How is ownership enforced on lists vs actions?** → lists: query-scoped by id; actions: post-load compare. F: "Which is safer?" → query scoping (no row even surfaces). T: Only knowing the compare pattern.
76. **What's the risk of returning entities directly?** → over/under-serializing, lazy-init exceptions, password-hash leak, tight coupling. F: "Where did it bite you?" → User in escrow JSON. T: DTO purism without admitting v1's leaks.
77. **Why GenerationType.IDENTITY — any downside?** → DB auto-increment; disables JDBC batch inserts (id needed after each row) — fine here; SEQUENCE for batching at scale. F: "UUID?" → no enumeration, bigger keys, index bloat. T: Not knowing batching implications.
78. **How would you add pagination to escrow lists?** → return Page<EscrowDTO> via findByBuyerId(Long, Pageable); default sort createdAt desc. F: "Cursor vs offset?" → cursor for deep pages. T: Paging without sort

stability.

79. **Explain read-committed vs repeatable-read in your flows.** → RC (default) fine: version predicate re-checks preconditions at UPDATE; RR would add snapshot consistency but not needed. F: "When RR?" → complex multi-read invariants. T: "Serializable everywhere."

80. **What is a lost update and where can't it happen here?** → overwrite of read-modify-write; can't happen on balances (never stored); guarded on escrow row by @Version. F: "Ledger?" → append-only, no target row. T: Saying "no lost updates anywhere" without the escrow-row nuance.

81. **How do you test concurrency?** → two threads/barriers in an integration test on real Postgres (Testcontainers), assert single payout; plus state-machine unit matrix. F: "Flaky?" → latch synchronisation + generous timeouts. T: Trusting H2 to reproduce PG locking.

82. **What's Hikari doing here?** → Spring Boot default pool to Neon pooler; sslmode=require. F: "Sizing?" → default 10; pooler endpoint multiplexes. T: Not knowing your pool exists.

83. **Why does LedgerController live under /api/auth/ — and why is that a problem?** → accident of naming; misleads route scanners and docs; requires auth (not permitAll) but isn't auth-related. F: "Rename?" → /api/ledger/transfer. T: Defending the path as intentional.

84. **What does spring.security.user.name/password do in your properties?** → Boot's default user config; inert once a custom SecurityFilterChain + your own filter own authentication. F: "Remove?" → yes, noise. T: Claiming it's your login.

85. **How would you version this API?** → /api/v1 prefix or header negotiation; breaking changes → v2 with overlap period. F: "Now?" → unprefixed (v1 implicit). T: Query-param versioning evangelism.

86. **What's your CORS setup exactly?** → SecurityConfig CorsConfigurationSource (5173, methods, headers, credentials) + MVC-level CorsConfig duplicate. F: "Why both?" → filter-level for security chain + MVC-level mapping; one would do — duplication noted. T: "CORS is server security" — it's browser* enforcement of client origin.

87. **How does the frontend know a request failed?** → response.ok check + readResponse handles JSON/text error bodies (because errors come as various shapes). F: "Improve?" → consistent error envelope from advice. T: Assuming always-JSON errors.

88. **Why does readResponse exist?** → backend returns JSON on success but plain text/JSON error depending on path; it sniffs content-type. F: "Root fix?" → unified error body. T: Not connecting it to the missing handler.

89. **Explain initiate's inline checkExpiry call.** → after save, immediately evaluates deadline; normally "Valid!!"; guards pre-expired edge (clock/tests). F: "Cost?" → one extra SELECT. T: Thinking it can expire a just-created escrow in normal flow (40s > 0s).

90. **What indexes would you add first and why?** → ledger_entry(account_id) — every balance read; escrow(buyer_id/seller_id) — lists; escrow(status) — scheduler. F: "Partial index?" → status='PAYMENT_INITIATED' keeps sweep tiny. T: "FKs are auto-indexed in Postgres" — false.

91. **What's the deadlock risk in your money paths?** → transfer locks nothing (inserts); version conflicts aren't deadlocks; risk would appear only with FOR UPDATE + opposite ordering. F: "Rule if you add locks?" → order by account id. T: Claiming deadlocks happen today.

92. **Why does confirmEscrow look up the seller's account via "ACC"+sellerId?** → account-number convention ACC{userId} from registration; deterministic. F: "Fragile?" → yes — convention-as-contract; a lookup by ownerId is safer. T: Pretending it's robust design.

93. **What happens if two escalations hit checkExpiry concurrently for the same id?** → both read PAYMENT_INITIATED; one EXPIRED commit wins; other → version conflict exception. F: "Harm?" → log noise only. T: Claiming double-expiry rows.

94. **How would you observe production behaviour?** → actuator health/metrics, structured logs, per-endpoint latency, ledger reconciliation job results, scheduler tick monitor. F: "Alert on?" → $\Sigma D \neq \Sigma C$, escrow stuck states, sweep failures. T: "println is my observability" — say you'd replace it.

95. **Why are some flows GET vs POST as they are?** → reads GET, state changes POST; no PUT/DELETE (no updates/deletes of resources by clients). F: "REST purity?" → confirm-delivery as POST action endpoint is acceptable pragmatic REST. T: Claiming full REST/Resource-based design.

96. **What does existsByEmail + save vs unique constraint teach?** → check-then-act races always need the DB constraint; constraint is truth, check is UX. F: "Generalise?" → every guard pair in the app (escrow holding exists, idempotency). T: Believing app checks replace constraints.

97. **How would you add an audit of who called the transfer endpoint?** → ledger rows only carry reference_id; add initiated_by column or separate operation log keyed by request id. F: "Reuse status log?" → that's escrow-scoped; ops need own trail. T: Claiming transfers are fully attributable today (they're not).
98. **Why is EscrowRepository.findById(long) (non-Optional) suspicious?** → shadows JpaRepository.findById with derived query returning entity; NPE-prone semantics vs Optional. F: "Fix?" → delete override, use inherited. T: Not noticing it in code review.
99. **What's your data-retention story?** → none implemented; payments need WORM-ish retention: stop deletes (already), archive partitions, PII policy on users. F: "GDPR delete vs ledger?" → anonymize user PII, keep money trail. T: "We can delete users" — FKs forbid (rightly).
100. **How do you keep the demo money honest?** → every rupee traces to ESCROW_HOLDING debits (seed) — total supply = Σ user balances + escrow-held; query proves it. F: "If it doesn't add up?" → reconciliation alarm. T: Hand-waving "test data".

23.3 ADVANCED (50)

101. **Redesign balances for scale — options and choice.** → (a) balance column + same-tx update, (b) periodic snapshot + delta scan, (c) incremental aggregate table, (d) cache+invalidate. Choice: (c) updated inside the transfer tx — keeps single-writer atomicity, O(1) reads, ledger remains truth. F: "Hot account contention?" → per-account sharded counters / queue aggregation. T: Redis-only truth.
102. **Prove (or disprove): a crash can never lose or orphan money in v1.** → Money paths are single-tx pairs (fund/release) → atomic; but registration seed is a multi-step non-tx flow → crash can orphan seed (user without account/money) — money exists but mismatch; and manual transfer has no balance rule so money can be created relative to policy (negative balances). Honest: conservation holds; policy doesn't. F: "Fix?" → tx + balance checks. T: Absolute "never".
103. **Design the idempotency store.** → table idempotency_keys(key PK, user_id, request_hash, status, response_json, created_at, expires_at); flow: INSERT (status=RECEIVED) → if conflict: read → if DONE return stored response; if RECEIVED return 409/retry-later; execute in tx; update row DONE+response in SAME tx. F: "TTL/cleanup?" → nightly purge after 48h. T: Storing response outside the business tx (race window).
104. **When would @Version be insufficient, and what then?** → cross-row invariants (total per account), non-versioned tables (ledger has none — but transfer policy spans accounts), or read-modify-write on non-versioned entity. Then: SELECT FOR UPDATE (@Lock(PESSIMISTIC_WRITE)), serializable txs, or constraint-based (balance >= 0 CHECK). F: "Which for balance ≥ 0?" → CHECK constraint + in-tx aggregate. T: Versioning the ledger (wrong tool).
105. **Make fundTransaction enforce balance atomically — write it.** → within the @Transactional fund: `BigDecimal bal = repo.derivedBalance(buyerAccountId); if (bal.compareTo(amount) < 0) throw new InsufficientFundsException();` — but derived-sum inside tx is a snapshot read; under RC two parallel funds can both pass → fix with (a) balance column + UPDATE accounts SET balance=balance-? WHERE id=? AND balance>=? conditional update, or (b) aggregate table row locked via FOR UPDATE. F: "Why is the naive check racy?" → phantom-free ≠ exclusive; both read old sum. T: Thinking a SELECT checks anything under concurrency.
106. **Where exactly would Serializable isolation break your app?** → scheduler sweep vs confirm, double release retries, and any two concurrent escrow actions would serialize → serialization failures surface as 500s; throughput collapses on escrow rows. F: "Better?" → keep RC + version. T: Upgrading isolation to fix design bugs.
107. **Walk through what PostgreSQL writes for a fund commit.** → INSERT ledger×2 + UPDATE escrow + INSERT log; WAL records, new tuples (MVCC), commit record → durability on flush. F: "Why can't half of it survive crash?" → no commit record → recovery ignores. T: Confusing undo logs (MySQL) with PG MVCC redo.
108. **Your status log and escrow status are in different txs in cancel — design the reconciliation.** → query: log rows whose chain doesn't end at current status → drift report; run post-deploy + nightly; auto-heal = reapply terminal transitions idempotently. F: "Prevent?" → @Transactional (one line). T: "Reconciliation is only for ledger."
109. **How would you horizontally shard this schema?** → shard by user/escrow id: users+accounts+ledger+escrows colocated per tenant; ESCROW_HOLDING becomes per-shard virtual (or

central treasury with two-phase entries); cross-shard release impossible if buyer/seller co-located by escrow id — choose escrow-id sharding; lookups by email need mapping directory or global secondary. F: "What breaks first?" → global pot semantics + unique email index. T: Sharding before measuring.

110. **Design at-least-once notification delivery off transitions without dup notifies.** → outbox row in same tx as transition; relay publishes with event id; consumer dedupes by event id (processed_events table). F: "Ordering?" → per-escrow ordering key. T: Publishing inside the tx (ghost events on rollback).

111. **How would you implement the auto-release job properly?** → query SHIPPED AND auto_release_at ≤ now() (indexed, batched, SKIP LOCKED via native query) → for each: buyer-ownership? no — SYSTEM actor → transition SHIPPED→RELEASED + transfer + log, per-escrow tx; ShedLock wrapper; metrics on lag. F: "SKIP LOCKED why?" → parallel workers without double-claim. T: Loading all SHIPPED rows each tick (unbounded).

112. **What's wrong with LocalDateTime here and how to migrate?** → no timezone; server clock locale; migration: Instant/timestamptz, store UTC, convert at edges; deadline math unchanged. F: "Data migration?" → assume old TZ constant, batch update. T: Mixing LocalDate/Timestamp types.

113. **JWT revocation design for logout/compromise.** → short access (5–15m) + refresh token store (hashed, rotated, revocable per user/session) or DB-checked jti denylist cache; middleware cost trade-off. F: "Why not 1h + nothing?" → stolen-token window. T: Denylist as eternal table (it's a TTL cache).

114. **Where does authorization belong in this codebase architecturally?** → URL rules for coarse authn; method security @PreAuthorize for role gates on services; object-level checks in domain services (they own context); consistent 403 vs business-409 mapping in advice. F: "Why not all in controller?" → bypass risk from other callers (scheduler internal calls skip HTTP). T: Scattering compares.

115. **Rate-limit login/register — design.** → bucket per IP+email (filter/bucket4j/Redis), lockout/backoff, generic error messages; also capture metrics. F: "Distributed?" → shared store (Redis) counters. T: Client-side throttling as security.

116. **Describe a full security review of v1 as findings with severities.** → Critical: transfer w/o ownership/balance; password hash serialization; secrets in repo; ADMIN self-registration. High: account data exposure; no rate limiting; 1h non-revocable tokens. Medium: error-code semantics; TRACE logging; show-sql; CORS * headers w/ credentials nuance. F: "First 3 fixes?" → §22.1. T: Only listing the pretty parts.

117. **How would you make initiate fully idempotent including concurrent same-key?** → §103 mechanism scoped to initiate; plus request-hash mismatch → 422; return 201 with stored escrow on replay; concurrent → 409 with Retry-After until first completes. F: "Key scoping?" → per-user unique (composite (user_id, key)). T: Global key namespace collisions.

118. **Argue append-only ledger vs event sourcing — same thing?** → Ledger = ES over money facts with account projections; I lack command rehydration/state rebuild via replay for escrow (that lives in the row) — partial ES. Full ES would make escrow state a projection too — powerful (free audit, temporal queries) but heavier. F: "When worth it?" → regulatory temporal queries. T: "They're unrelated."

119. **Hot-row problem: one seller, thousands of simultaneous releases — trace and fix.** → releases touch distinct escrow rows (version per row — fine) but aggregate table/balance row would serialize; fix: per-account write sharding (N sub-buckets), async settlement batch, or queue per account. F: "Does ledger itself contend?" → insert-only: no. T: Blaming the ledger table.

120. **Design reconciliation end-to-end.** → daily job: (1) $\Sigma D = \Sigma C$ global; (2) per-account statement recompute vs cached; (3) ESCROW net vs Σ active escrows; (4) log-chain vs status; (5) entry_id gap report (rollbacks are normal — flag only anomalies); output report + alert, freeze money ops on critical mismatch. F: "During traffic?" → snapshot tx / read replica. T: Assuming gaps mean corruption.

121. **What changes with 10k RPS on /api/me?** → balance full-scan dies first; add aggregate read (O(1)); cache with ledger-version invalidation; then DB read replica; paginate ledger. F: "Cache coherence?" → bump version key in the same tx as ledger writes. T: Caching before fixing O(n).

122. **Implement optimistic-lock retry properly.** → retry only idempotent ops; for release: re-read state → if already RELEASED → return success (business idempotency) else retry whole tx (new read) with backoff + jitter, max N; never retry blindly non-idempotent sends. F: "Spring support?" → @Retryable + careful tx boundaries (retry outside tx). T: @Retryable inside the tx (same snapshot forever).

123. **Why can't you just synchronize (Java) the release method?** → multiple JVM instances; sync guards memory not DB; DB truth needs DB mechanisms. F: "Single instance then?" → still wrong: sync serializes wrongly + doesn't cover other writers (scheduler, SQL console). T: Reaching for synchronized in

interviews.

124. Trace a request through Tomcat threads — where can thread-local state leak? →

SecurityContext is thread-local, cleared per request by the filter chain; async servlets need context propagation; @Scheduled threads have no auth context (scheduler actor=null reflects that ●). F: "Why does the scheduler pass null actorId?" → no authentication on that thread — deliberate. T: Reading SecurityContext in scheduler code.

125. Your @Transactional self-invocation pitfall — any occurrences? →

transactionRequest calls this.checkExpiry(...) — if checkExpiry were @Transactional, self-call would bypass the proxy; today harmless since it isn't ●. F: "General rule?" → externalize self-called tx methods or use TransactionTemplate. T: Claiming @Transactional works via this-calls.

126. How would you structure domain events from the transition service? →

collect events in-tx, AFTER_COMMIT listener publishes; store outbox for reliability; consumers idempotent by event id. F: "Why not publish directly?" → ghost events on rollback / notification without commit. T: Event-in-constructor patterns.

127. PG tuple bloat from status updates — mitigate? →

escrow row updated ~5x (hot updates, new tuples), autovacuum handles; wide rows with TOAST could bloat: keep row narrow, fillfactor tuning, monitor. F: "Ledger?" → insert-only, no churn. T: Fearing updates that are few.

128. Design multi-currency. →

ledger rows add currency column (ISO-4217), accounts single-currency (no mixing in one account), exchange via FX entries between currency accounts (both sides same value in base), amounts scaled integers (paise/cents). F: "Rounding?" → banker's rounding at FX only, never per-entry. T: One account multi-currency.

129. Schema-migration strategy from ddl-auto to Flyway on a live DB. →

baseline against current schema → validate mode in app → forward-only migrations in review; expand-contract for renames (add column → dual-write → backfill → switch → drop). F: "Hotfix rollback?" → backward-compatible migrations only. T: ddl-auto=update in prod.

130. Backpressure: what happens under burst at initiate? →

Tomcat queue grows → Hikari pool (10) saturates → connection wait timeouts → 500s; fix: limit concurrent via semaphore/bulkhead, queue at LB, timeouts sane, and idempotency makes client retries safe. F: "First knob?" → pool size + request timeouts. T: Unlimited threads as resilience.

131. What would you monitor on the ledger table specifically? →

write rate, $\Sigma D = \Sigma C$ drift check, entry_id gap velocity (rollback storms), largest accounts movement anomalies, replica lag if reads move. F: "SLO?" → reconciliation green daily + drift alert <5min. T: Only CPU/RAM dashboards.

132. Security of the Idempotency-Key header itself — abuse vectors? →

user A spoofing user B's key namespace (scope keys per user), unbounded key cardinality (rate-limit + length cap), key leakage in logs (PII-ish). F: "Per-user uniqueness?" → composite unique (user_id, key). T: Global unique(key) only (matches v1's single-column constraint — note the improvement).

133. Explain why transfer needs no row locks but a balance>=0 policy needs one. →

inserts append; no precondition on existing state. Policy reads state to gate write → read-then-write race → conditional UPDATE or lock makes precondition+write atomic. F: "CHECK constraint alternative?" → DB enforces at write; still needs balance materialized. T: "Constraints fix everything" — only materialized ones.

134. Walk through 2 instances + Neon pooler: what subtle bugs appear? →

PgBouncer transaction-mode breaks session-level features (prepared statements misconfig, set-based session vars), clock skew on deadlines, duplicate sweeps (safe via version). F: "Pooler mode?" → transaction pooling; use statement_timeout accordingly. T: Assuming pooler = plain connection.

135. Design admin dispute resolution with proper authz. →

ADMIN role gated by @PreAuthorize at endpoint + server-issued admin provisioning (no self-register), decision endpoint: RESOLVE release|refund → same tx (transfer+transition+dispute status) as \$22 refunds; full audit with admin actorId + note. F: "Dual control?" → two-admin approval table for high amounts. T: Trusting frontend role hiding.

136. Where would you put a fee model in the ledger? →

third entry pair per release: ESCROW→PLATFORM_FEE (fee account SYSTEM), seller gets amount-fee; still $\Sigma D = \Sigma C$; fee policy in service config. F: "Split transfer?" → extend transfer() to N legs atomically. T: Rounding fees per-entry drift.

137. Justify your entity graph: why no @OneToMany collections? →

avoid in-memory list loading & cascade surprises; queries go child→parent via derived finds; simpler serialization. F: "Cost?" → no convenience traversal; fine at this scale. T: Mapping everything bidirectionally "for completeness".

138. **What does `ddl-auto=update` never do, and why is that dangerous?** → never drops columns/renames; drift accumulates; combined with manual console edits, schema truth diverges from entities; prod = validate + migrations. F: "Detect drift?" → schema diff in CI. T: "update is safe because it never deletes" — silent divergence is the danger.
139. **Design contract tests between your React app and API.** → OpenAPI spec from controllers (springdoc) → generated TS client → contract tests in CI; error envelope fixed so `readResponse` simplifies. F: "Now?" → hand-synced `api.js` + formatters. T: Snapshot-testing JSON with entity leakage baked in.
140. **Which single query would you EXPLAIN first and what do you expect?** → `findByAccountId` balance loop: Seq Scan on `ledger_entry` (no index) → add index(`account_id`) → index scan; then measure `/api/me` p95. F: "After index?" → row-count still $O(\text{history})$ → aggregate. T: Optimizing before EXPLAIN.
141. **How do you keep the scheduler from starving long GC/DB blips?** → `fixedRate` skipped executions are coalesced; make sweep idempotent + bounded batch; metrics on last-success timestamp; external trigger fallback. F: "Catch-up semantics?" → document `fixedRate` vs `fixedDelay` choice. T: Assuming missed ticks queue up.
142. **What breaks if two escrows share an idempotency key across DIFFERENT users (today)?** → global unique column → innocent second user gets 500. F: "Correct model?" → (`user_id`,key) composite; that's the §132 fix. T: Defending global uniqueness.
143. **Describe zero-downtime deploy of a change to transition rules.** → backward-compatible window: new code accepts old+new states, feature-flag new edge; deploy; drain old states; tighten. With single binary: expand-contract on status values + data backfill. F: "Terminal-state addition?" → rollout with read-side tolerance first. T: Hard enum cutover.
144. **Your JWT has no kid. Rotating the secret with zero downtime?** → dual-key verification window: verify with new, fall back old; mint new-only after TTL expires old tokens; or kid header map. F: "Compromised now?" → immediate rotate + forced expiry via short TTL/revoke. T: Single-key hot swap.
145. **Where is the trust boundary in this system, precisely?** → everything after the TLS termination: client body (validated), headers (token verified signature-only — role claim NOT trusted for authz ●), path ids (ownership-checked per object), scheduler (trusted internal), DB (trusted). F: "Weakest boundary?" → the unauthenticated surfaces (register role field). T: Trusting the token role claim.
146. **How would you add an approval step for high-value releases?** → state between SHIPPED and RELEASED (PENDING_APPROVAL) with new edges; policy in service; dual-control rows; notify; SLA sweep. F: "Machine change cost?" → enum + `isValid` edges + backfill — contained by design. T: Implementing approval outside the state machine.
147. **Compare @Scheduled vs Quartz vs external cron calling an endpoint.** → @Scheduled: zero infra, single-node semantics; Quartz: clustered locks, persistence, misfire policy; external cron + admin endpoint: simplest multi-node (auth!). For v2 auto-release: Quartz or ShedLock+@Scheduled. F: "Why not Kafka?" → overkill for 2 jobs. T: Distributed cron without auth.
148. **What invariant would you add as a DB CHECK constraint right now?** → CHECK (`amount > 0`) on `ledger_entry` and `escrow_transaction`; CHECK (`buyer_id <> seller_id`) on `escrow`; CHECK (`status IN (...)`). F: "Why not FK-level everything?" → some invariants span rows → app or triggers. T: Trigger-everything architecture.
149. **Your POST /api/accounts/new binds an entity — full attack surface?** → mass assignment (ownerType SYSTEM + arbitrary accountNumber, overwriting pot name blocked only by unique), no authz, no validation. F: "Fix?" → kill endpoint or DTO + role gate. T: "It's just a demo endpoint" — attackers don't read intentions.
150. **Explain the difference between your two InvalidTransitionException classes.** → `escrow` package: real exception thrown by transition; exception package: empty stub shadow-name — cleanup hazard (wrong import compiles? no — different packages, but confusing review). F: "Rule?" → one exception home (`com.revertpay.exception`) + advice. T: Not noticing duplicates in your own tree.

23.4 BRUTAL FOLLOW-UPS (30)

151. **"Show me exactly where money could be created from nothing."** → POST `/api/auth/transfer` credits any account with no balance rule ●; and seeds mint (intended). Policy vs conservation distinction wins points.

152. **"Your ledger proves $\Sigma D = \Sigma C$ — so no money vanished. Yet a user lost money. How?"** → policy violations: negative balances via transfer/fund; ownership breach. Conservation $\neq$ correctness of policy.
153. **"Why is @Version not on LedgerEntry?"** → append-only: no updates → nothing to lose; version guards mutable state (escrow row). F: "What if entries became editable?" → then immutability model broken; version + audit trigger.
154. **"Two @Transactional methods call each other — how many tx?"** → REQUIRED joins → 1; REQUIRES_NEW would suspend → 2; inner rollback marks outer rollback-only. Map to fund→transfer (1 tx ●).
155. **"Walk your JWT from fetch() header to SecurityContext — byte level."** → base64url segments; HMAC over A.P; verifyWith(secret); parseSignedClaims; sub→Long; DB load; authority list. (§15.3.)
156. **"If I steal a token right now, what's your blast radius and TTL?"** → full account acting rights, 60 min max, no revocation, no refresh; mitigations to ship: short TTL + refresh + jti denylist + anomaly logout.
157. **"Your scheduler and confirm race — give me the exact winner matrix."** → both read PI: confirm-wins → CREATED (+scheduler exception in logs); expire-wins → confirm throws InvalidTransitionException 500; neither half-applies. No money involved.
158. **"Why is your exception handler empty a security issue, not just UX?"** → raw messages leak internals (class names, SQL fragments via DataIntegrityViolation chains), inconsistent codes hamper client logic, stack traces to stdout. Blueprint: advice + generic bodies + log-side detail.
159. **"Prove release cannot double-pay — not by @Version hand-waving, by isolation reasoning."** → RC: T2's UPDATE re-evaluates against latest committed row (version=4) → predicate false → 0 rows → StaleObjectState → rollback including INSERTs (same tx). Version predicate = optimistic compare-and-swap at the storage layer.
160. **"What if two nodes have clock skew and the same escrow's deadline?"** → deadline stored, compared per-node with its clock: node +3s expires later; window wobble $\leq$ skew; no correctness break for money (expiry pre-funds). Fix: DB now() for critical math.
161. **"Why does registration not being atomic matter if seeds are just demo money?"** → pattern risk: real money flows will copy the shape; also broken onboarding states (user w/o account 500s on /api/me forever). Engineering principle over rupee value.
162. **"Your password hashes leak in JSON — how did that happen architecturally?"** → entities as response bodies + class-level @Getter; no DTO boundary on list endpoints. Fix + retro: serializer allowlist, API contract tests asserting absence of password.
163. **"Design the fix for ADMIN self-registration without downtime."** → stop trusting client role: default BUYER; admin grant via migration/CLI; existing ADMINS preserved by data backfill; add @PreAuthorize on future admin routes; invalidate old tokens optionally (TTL $\leq$ 1h anyway).
164. **"Someone POSTs /{id}/fund 10,000 times/second. Trace every layer."** → Tomcat accepts → filter+DB per request (user load) → pool exhausts → connection wait timeout → 500 storm; DB write rate if pool larger; state machine rejects after first success — CPU burn; fix: rate limit per user+escrow, idempotency-key on fund, circuit breaker.
165. **"Which endpoint would you delete today, and why?"** → POST /api/accounts/new (pure liability) + Bruno/Testy toys; then gate transfer.
166. **"EscrowStateLog grows forever — at 1B rows what breaks?"** → index bloat, vacuum cost, backup time; partition by month (PK needs partition key), archive cold partitions to S3, keep recent hot. Queries unaffected with partition pruning by timestamp.
167. **"Justify trusting Authorization header parsing substring(7)."** → brittle but standard for "Bearer "; robust: parse scheme case-insensitively, 401 on malformed. Show awareness: v1 assumes exact prefix.
168. **"Why is your optimistic-lock failure a 500 to the client — what SHOULD the client do?"** → currently can't distinguish conflict vs bug; should: 409 + Retry-After/state body; client re-GETs escrow → sees RELEASED → treats as success (business idempotency).
169. **"Where exactly do you trust path variables, and what's the pattern that makes that safe?"** → id → load → compare owner OR scope query by owner; pattern: never act on a loaded object without the ownership predicate in the same service call. Name the endpoints that violate it (accounts/transfer).
170. **"Your state log's reason is a free-text string — audit risk?"** → uncontrolled semantics; make it enum + optional note column; free text allowed only in dedicated field with length cap; log injection via control chars — sanitize.

171. **"Implement the refund in 10 minutes — what breaks first?"** → missing buyer account lookup on escrow (fund knows buyer; refund needs `ACC{buyerId}` — same convention), no admin authz, no idempotency on refund → double-refund blocked only by terminal REFUNDED (good), status log reasons ad hoc; dispute stays OPENED forever (no RESOLVED path wired).
172. **"How would an attacker abuse the 40-second confirmation window?"** → squatting: mass-initiate to a seller to spam; no cost. Rate-limit initiate per user; require signed order draft. Money-safe regardless.
173. **"Your balance derivation — someone transfers ₹0.005. What happens?"** → BigDecimal stores it (NUMERIC default scale ●); INR formatter shows ₹0.00 (display rounding, value intact); policy: pin scale=2 + CHECK; smallest unit = paise integer.
174. **"The frontend stores JWT in localStorage — defend AND attack it."** → Attack: any XSS exfiltrates; defense: CSP, sanitize, minimal token lifetime; alternative httpOnly cookie + CSRF double-submit; explain why header-Bearer was chosen for v1 simplicity.
175. **"What's your RTO/RPO if Neon loses the region?"** → v1: none configured (honest); design: PITR + cross-region replica, RPO ≤ replication lag (~seconds), RTO = promote + redeploy (~minutes), plus reconciliation before unfreeze. The app holds no state → DR is a DB problem — say that proudly.
176. **"One of your ledger rows is manually UPDATED by a DBA. What detects it?"** → today: nothing (no hashes/audit triggers) ●; add: append-only grants, trigger-based audit table, hash chain, and reconciliation can't catch single-pair tamper — that's why immutability must be enforced at DB permission level, not convention.
177. **"Why is version BIGINT and does incrementing forever matter?"** → Long, 63 bits, no realistic overflow; per-row updates only at transitions (≈5/escrow). If you batch-update hot rows thousands/sec, still centuries. Know the number.
178. **"Give me the exact SQL your findByIdempotencyKey would generate."** → `SELECT ... FROM escrow_transaction WHERE idempotency_key=?` — unique index makes it index scan; the point: it exists, it's cheap, it's just never called (own it with a smile).
179. **"If I open 5 browser tabs and click 'Confirm' simultaneously on the same escrow — walk it."** → 5 concurrent POSTs; exactly one commits RELEASED (version); others: either invalid-transition (sequential arrivals after first commit → clear 409-after-fix) or optimistic-conflict (true parallels); UI shows error toasts today — improvement: poll/re-fetch and show 'already released'.
180. **"Final: convince me you own this codebase."** → Deliver the 5-point honest-gap list + the 1-day fix plan + the one thing you're proud of (transactional double-entry release with version-guarded single payout) + the first test you'd write (state machine matrix). Calm, specific, zero bluff.
-

24. FINAL REVERTPAY CHEAT SHEET (≤10 revision pages)

Page 1 — Identity

- **What:** escrow payments app. Money parked in system pot ESCROW_HOLDING until delivery confirmed.
- **Stack:** Spring Boot **4.1.1**, Java **25**, Spring MVC + Data JPA + Security + Validation, jjwt **0.13** (HS256), PostgreSQL **Neon** (sslmode=require), React **19** + Vite **8** (fetch, no router), token in localStorage.
- **Repo:** Revertpay/backend + Revertpay/frontend; single commit; no README/migrations/CI.

Page 2 — Architecture flow (recite)

React fetch (Bearer JWT) → Vite proxy /api→:8080 → **JwtAuthenticationFilter** (verify sig+exp → load user from DB → ROLE_X into SecurityContext) → **SecurityFilterChain** (STATELESS, CSRF off, permitAll: register/login; rest authenticated) → Controller (extract token) → DTO @Valid → **Service** (rules + @Transactional) → Repository (derived queries) → Hibernate (ddl-auto=update) → PostgreSQL. **Money services:** LedgerService.transfer = doDebit+doCredit, @Transactional — the only way money moves.

Page 3 — Lifecycle + endpoints

Endpoint	Actor	State change	Money
POST /api/escrow/initiate (+Idempotency-Key)	buyer	→ PAYMENT_INITIATED (deadline +40s)	none
POST /{id}/confirm	buyer	PAYMENT_INITIATED→CREATED	none
POST /{id}/cancel	buyer	PAYMENT_INITIATED→CANCELLED	none
POST /{id}/fund	buyer	CREATED→FUNDED	buyer→ESCROW
POST /{id}/ship	seller	FUNDED→SHIPPED (+shippedAt, autoReleaseAt+5m)	none
POST /{id}/confirm-delivery	buyer	SHIPPED→RELEASED	ESCROW→seller
POST /{id}/dispute	buyer	SHIPPED→UNDER_REVIEW (+Dispute OPENED)	frozen
GET /api/escrow/buyer · /seller	self	read	read
scheduler (5s)	system	PAYMENT_INITIATED→EXPIRED (past deadline)	none
POST /api/auth/transfer	any authed	—	any→any ⚠
Terminal: CANCELLED, EXPIRED, RELEASED, REFUNDED. Every transition → escrow_status_log row (from,to,actor,reason,ts).			

Page 4 — Ledger in one breath

Append-only ledger_entry(account FK, DEBIT/CREDIT, BigDecimal amount, reference_id, createdAt). Balance = Σcredits−Σdebits, derived (UserController /api/me, AccountController /balance) — **no balance column**. Invariant ΣD=ΣC by construction (every movement = one atomic pair). Registration mints ₹5000:

D:ESCROW_HOLDING C:ACC{id} (treasury goes negative — expected). Fund: D:buyer C:ESCROW ref=escrowId. Release: D:ESCROW C:seller ref=escrowId. Reconciliation: ESCROW net == $\sum$ escrows in FUNDED/SHIPPED/UNDER_REVIEW.

Page 5 — Transactions & concurrency

@Transactional: LedgerService.transfer, EscrowService.fundTransaction, EscrowService.confirmEscrow ✓. NOT: register, initiate, confirm, cancel, ship, dispute, checkExpiry △ (log/save drift + register partial-state risks). Concurrency doctrine: state rules = in-memory isValid(); commit-time truth = @Version on escrow_transaction (WHERE version=?; 0 rows → ObjectOptimisticLockingFailureException → rollback). Two releases → exactly one payout (loser's ledger rows roll back with its tx). Race confirm-vs-expire → version decides; money never involved at that stage. No @Lock/FOR UPDATE anywhere; Postgres RC isolation; DB uniques as final guards. Append-only ledger = no lost updates on money.

Page 6 — Idempotency (honest)

Present: header required at initiate ●; idempotency_key unique+NOT NULL on escrow ●; DB rejects duplicate-key insert. Missing ●: replay lookup (findByIdempotencyKey never called), requestHash unused, client regenerates key per click (retry → new key → dup escrow), keys absent on fund/ship/release. De-facto: state machine blocks money replays (2nd fund/release → InvalidTransitionException → 500). Fix: idempotency_keys table (key PK per user, request_hash, response snapshot, same-tx DONE) or at minimum replay-return existing escrow; reuse one key per logical operation client-side.

Page 7 — Security & JWT

JWT: HS256, secret from properties (committed △), claims sub=userId/email/role, 1h exp. isValid = signature (parseSignedClaims throws) + explicit exp check; catch-all false. Filter loads user **from DB** → authority ROLE_<DB role> (token role claim not trusted for authz ●). Authorization: URL-level anyRequest().authenticated(); role + ownership checks **inside services** (token userId vs escrow.buyer/seller). No @PreAuthorize, no hasRole rules ●. BCrypt encode/matches. STATELESS, CSRF off (Bearer-only — correct; revisit if cookies). CORS 5173. Gaps to volunteer ●: account endpoints + transfer lack ownership/balance checks; ADMIN self-registerable; **User entities (with password hash) serialized in escrow lists**; no refresh/revocation; no rate limiting; secrets in git.

Page 8 — JPA + PostgreSQL quick facts

6 entities: User(users; email unique), Account(accounts; account_number unique; ownerType USER/SYSTEM), LedgerEntry(ledger_entry), EscrowTransaction(buyer_id, seller_id FK; amount; status; confirmationDeadline; idempotency_key UNIQUE; requestHash unused; @Version; shippedAt/autoReleaseAt/resolvedAt(resolvedAt never set)), EscrowStatusLog, Dispute. All IDs IDENTITY; enums @Enumerated(STRING); @PrePersist createdAt on Account/LedgerEntry; ManyToOne default EAGER (N+1 on lists); no @OneToMany, no cascade, no @Query/JPQL, no @Lock. PG: ddl-auto=update (→ Flyway in prod); FKs not auto-indexed → add indexes: ledger_entry(account_id), escrow(buyer_id, seller_id, status partial PI); BigDecimal→NUMERIC (pin precision/scale); LocalDateTime→timestamp (→ UTC timestamptz).

Page 9 — Scheduler

@EnableScheduling + EscrowScheduler @Scheduled(fixedRate=5000) → all PAYMENT_INITIATED → checkExpiry → past confirmationDeadline(40s) → EXPIRED (actorId=null). Restart-safe (deadline in DB). Single-node assumption; multi-instance → version-guarded duplicate sweeps (safe, noisy → ShedLock). Per-item exception aborts that tick's loop. **EscrowAutoReleaseJob = empty stub ● — autoReleaseAt stored/displayed, never enforced.**

Page 10 — Your honest-gap script (memorize verbatim-ish)

"v1's money core is solid: atomic double-entry transfers, a centralized state machine with full audit logging, optimistic-lock-guarded escrow updates so concurrent releases can't double-pay, and JWT auth with

per-object ownership checks on every escrow action. The gaps I'd fix first, in order: (1) serialize users through DTOs — right now escrow lists expose the password hash; (2) the manual transfer endpoint needs ownership + balance enforcement; (3) a real exception handler mapping domain errors to 401/403/404/409 instead of 500s; (4) `@Transactional` on the remaining state-change methods; (5) wire the declared-but-unused idempotency replay and persist a stable client key per logical payment; (6) build the auto-release job against `autoReleaseAt`; (7) tests — starting with the 81-pair state-machine matrix and a two-thread release race test on Testcontainers Postgres. I can defend every line that exists — and every line that doesn't."

25. FINAL RULE

After this handbook you should be able to:

1. Open **any** file in Revertpay/backend or frontend/src and explain what it does, who calls it, and what breaks if it fails — because §3–§4 covered every non-stub class and §20 walked the 20 critical blocks.
2. Say "● that's in the code — here's the file and the line of reasoning", and — more importantly — say "● **that's not in the supplied implementation; here's what v1 does instead, and here's my design to add it.**"
3. Answer the money questions (§7, §9, §10) as demonstrations — walking the two-release race table and the ledger sums on the whiteboard — not as memorized paragraphs.
4. Survive follow-ups (§23) because every answer carries its own trap-warning.
5. End every architecture answer the same honest way: **actual code > generic theory, understanding > memorization, honest defense > exaggeration.**

Handbook generated from the supplied RevertPay-v-1.0 source tree (commit d5b2dd7). Every ● claim is traceable to a file in that tree; every ● was verified absent by inspection and grep.